Integer Anchored Densest Subhypergraph Search via Reusable Orientations

Qi Zhang, Zhou Zhou, Jie He, Xi Guo, Xiaotong Zhang, Xu-Cheng Yin

University of Science and Technology Beijing, Beijing, China;
qizhangcs@ustb.edu.cn; 2006265114zz@gmail.com; hejie@ustb.edu.cn; xiguo@ustb.edu.cn;
zxt@ies.ustb.edu.cn; xuchengyin@ustb.edu.cn

***Abstract*—Anchored densest subhypergraph search identifies dense high-order structures that contain mandatory query vertices while favoring a reference set. Existing formulations require users to select a locality parameter. Their exact solvers repeatedly perform flow computations at successive density thresholds, even on a static hypergraph, and provide no incremental maintenance: a single hyperedge insertion or deletion triggers a full recomputation. To address these limitations, we study exact search and dynamic maintenance for the integer anchored densest subhypergraph problem, defined using a simpler, parameter-free integer R-subhypergraph density, where each retained hyperedge contributes two reward units and each selected non-reference incidence incurs one penalty unit. We establish an exact anchor-bank orientation characterization and derive a min–max characterization of the optimal integer density. Building on this characterization, we develop CB-Global and CB-Local, two Ceil-Balance algorithms that optimize adjusted owner loads and compute complete residual closures at the optimal boundary. Both algorithms return a set that maximizes total excess at that boundary and contains every real-valued optimal anchored subhypergraph. CB-Local reuses a query-independent orientation index. For dynamic maintenance, our Dynamic-Path algorithm maintains an upper-bound orientation and a maximum-excess solution, restoring both after each update via residual-path repair. Extensive experiments and case studies demonstrate the efficiency, scalability, and effectiveness of our solutions.**

I. Introduction

Graphs represent complex systems through pairwise relations [1]. Many real interactions, however, involve groups rather than pairs [2], [3]. For example, a shopping basket contains multiple products, an online discussion involves several users, and a scientific paper connects multiple authors. Modeling these interactions using only pairwise edges loses their group semantics. Hypergraphs address this limitation by representing each group interaction as a hyperedge, that is, a set of vertices that may contain more than two members [4], [5]. By preserving group semantics, hypergraphs provide a natural foundation for analyzing cohesive higher-order structures.

The densest-subhypergraph problem is central to identifying cohesive groups in hypergraphs [6], [7]. It seeks a vertex set that maximizes the ratio of internal hyperedges to selected vertices and serves as the hypergraph counterpart of the classical densest-subgraph problem [8], [9], [10]. However, the standard formulation is query-independent: it returns a globally densest set without considering query-specific entities. It therefore cannot directly support query-centered tasks such as recommending products related to prior purchases, discovering discussion communities around selected users, or identifying collaboration groups around designated researchers.

To support these applications, Anchored Densest Subgraph (ADS) search was introduced on graphs as a query-aware variant of the densest-subgraph problem [11], [12]. A query specifies a reference set R of preferred vertices and, when required, a mandatory anchor set $A \subseteq R$ that every answer must contain. ADS biases the result toward R by penalizing selected vertices outside it. Huang et al. extend this idea to hypergraphs under all-pins semantics [6]. For a set S, let $m(S)$ denote the hyperedges contained in S, and let $\mathrm{Vol}(S \setminus R)$ count the hyperedge incidences of its non-reference vertices. Their Anchored Densest SubHypergraph (ADSH) objective maximizes $\rho_R^{\epsilon}(S) = q_\epsilon(S)/|S|$, where $q_\epsilon(S) = m(S) - (\epsilon/2)\,\mathrm{Vol}(S \setminus R)$, subject to $A \subseteq S$ when anchors are required. Thus, internal hyperedges reward cohesion, whereas the volume term discourages expansion beyond R. The locality parameter $\epsilon \geq 0$ controls this tradeoff: at $\epsilon = 0$ the penalty vanishes, while a larger ϵ confines the result more strongly around R. To solve ADSH, Huang et al. develop HGV-Global for $\epsilon \geq 0$ and HGV-Local for the regime $\epsilon \geq 1$.

However, existing ADSH methods have three limitations. First, exact optimization requires repeated maximum-flow computations at successive density thresholds. Whenever a denser set is found, its density becomes the next threshold: HGV-Global solves a complete flow network at each threshold, whereas HGV-Local may solve several progressively expanded local networks. Second, both methods are static and provide no incremental maintenance. After a hyperedge insertion or deletion, the previous answer no longer certifies optimality, so the solver must rerun on the updated hypergraph. Third, the formulation requires users to choose ϵ, which determines the penalty outside R; different choices may therefore produce different subhypergraphs for the same query.

To provide fixed locality semantics while exposing reusable discrete structure, we define the *integer R-subhypergraph density* of S as $\bar{\rho}_R(S) = \lceil (2m(S) - \mathrm{Vol}(S \setminus R))/|S| \rceil$. Given a hypergraph $\mathcal{H}$, a reference set R, and mandatory anchors A, the *Integer Anchored Densest Subhypergraph* (I-ADSH) problem seeks a set $S \supseteq A$ maximizing this score. The objective assigns two reward units to each internal hyperedge and one penalty unit to each selected incidence of a vertex outside R, eliminating the need for a user-selected locality parameter. For every S, $\bar{\rho}_R(S) = \lceil 2\rho_R^1(S) \rceil$; consequently, the

optimum I-ADSH value is the ceiling of twice the optimum ρ_R^1 value. On rank-two hypergraphs, I-ADSH degenerates exactly to the Approximate Anchored Densest Subgraph (AADS) objective on graphs [13]. Crucially, the integral level admits a direct token-balancing formulation for static optimization and a bounded candidate window after a hyperedge update. These properties enable exact computation and dynamic maintenance without rerunning a static optimizer.

These properties yield a unified orientation framework for exact static computation and dynamic maintenance. Each hyperedge contributes two tokens, assigned either to incident non-anchor vertices or to a shared anchor bank. Owner budgets encode a tested density level: a feasible assignment certifies that no anchored subhypergraph reaches that level, whereas a deficient assignment exposes an anchored witness. Ceil-Balance directly minimizes the maximum adjusted owner load. We instantiate it in two algorithms, CB-Global and CB-Local: CB-Global materializes the query-specific orientation, whereas CB-Local reuses a query-independent orientation index and stores only sparse query-specific changes. At the resulting integer level k, it processes all remaining positive-excess roots and returns a set maximizing $N_R(S)-(k-1)|S|$. This output contains every real-valued ADSH optimum at $\epsilon = 1$. For dynamic maintenance, Dynamic-Path preserves this containment guarantee together with an orientation certifying that level $k+1$ is unattainable. In summary, we make the following contributions.

A parameter-free integer ADSH formulation. We formulate I-ADSH using two-unit hyperedge rewards and unit penalties for non-reference incidences. Its optimum is the ceiling of twice the optimum ADSH density at $\epsilon = 1$, and its rank-two specialization is the AADS objective. We prove that, at the optimal integer boundary, the returned set maximizes the aggregate excess above the threshold and contains every real-valued optimum. This result characterizes the returned set while allowing a density gap below one in the two-unit scale.

Exact static computation with Ceil-Balance. We establish an anchor-bank threshold characterization and a min–max formulation of the optimum level. CB-Global constructs the query-specific orientation, balances owner loads, and collects complete residual closures from all boundary roots. CB-Local performs the same operations using a shared index and sparse query corrections. Closed regions are retained while the remaining roots are processed. We prove that the final closure attains a universal excess upper bound, thereby establishing both integer optimality and optimal-core containment.

Dynamic maintenance via residual-path reorientation. We prove that a single hyperedge update changes the optimum integer level by at most one, restricting it to the current or adjacent level. Exploiting this bound, Dynamic-Path uses residual-path reversals to update the stored orientation and determine the new level, and extracts the complete residual closure from owners with load above budget at the new optimum boundary. It preserves the upper-bound orientation and avoids materializing a maximum-flow network.

Extensive experiments. We evaluate our algorithms for the I-ADSH problem on seven real-world hypergraphs. Our results show that: *(i)* CB-Global is 21.61–$111.46\times$ faster than HGV-Global and 11.57–$74.30\times$ faster than HGV-Global-I, while CB-Local is 3.59–$65.25\times$ and 1.85–$22.63\times$ faster than HGV-Local and HGV-Local-I, respectively; *(ii)* the reusable CB-Local index adds only 3.03–242.19 MB of retained state and incurs modest construction overhead while enabling substantial query-time gains; *(iii)* CB-Local remains faster than both HGV-Local variants across the tested reference-set sizes and hypergraph densities, demonstrating scalability; and *(iv)* Dynamic-Path is roughly two–four and three–five orders of magnitude faster than CB-Local and HGV-Local-I, respectively, across random-global, query-local, and temporal updates. We further show that I-ADSH closely matches ADSH in R-subhypergraph density and that I-ADSH represents a larger optimal anchored team than ADSH in a case study. For reproducibility, the source code of this paper is released at https://github.com/zhouzhou5114/I-ADSH.

II. Preliminaries

Consider an unweighted and undirected hypergraph $\mathcal{H} = (V, \mathcal{E})$, where V is the set of vertices and $\mathcal{E}$ is the set of hyperedges. Each hyperedge $e \in \mathcal{E}$ is a set of at least two vertices. For a vertex set $S \subseteq V$, let $\mathcal{E}[S] = \{e \in \mathcal{E} \mid e \subseteq S\}$ denote the set of hyperedges induced by S, and let $m(S) = |\mathcal{E}[S]|$. This definition follows the all-pins semantics: a hyperedge contributes to $m(S)$ only when all of its vertices belong to S. For each vertex $v \in V$, define its hypergraph degree by $d_{\mathcal{H}}(v) = |\{e \in \mathcal{E} \mid v \in e\}|$, which counts incident hyperedges rather than distinct neighboring vertices. The volume of a vertex set $T \subseteq V$ is $\mathrm{Vol}_{\mathcal{H}}(T) = \sum_{v \in T} d_{\mathcal{H}}(v)$, where vertex degrees are evaluated in the current hypergraph $\mathcal{H}$. We use $r = \max_{e \in \mathcal{E}} |e|$ to denote the rank of $\mathcal{H}$. When the underlying hypergraph is clear, we write $d(v)$ and $\mathrm{Vol}(T)$ for short.

We first recall the existing ADSH formulation.

Definition 1: (Parameterized R-Subhypergraph Density) Given a hypergraph $\mathcal{H} = (V, \mathcal{E})$, a reference vertex set $R \subseteq V$, and a locality parameter $\epsilon \geq 0$, the parameterized R-subhypergraph density of a nonempty vertex set $S \subseteq V$ is defined as follows [6]:

$$\rho_R^{\epsilon}(S) = \frac{m(S) - (\epsilon/2)\,\mathrm{Vol}(S \setminus R)}{|S|}. \tag{1}$$

Huang et al. [6] formulate ADSH without mandatory anchors in their definition, but note that their analysis extends to a prescribed anchor set; since our queries require mandatory anchors, we use this extension throughout our study.

Definition 2: (Anchored Densest SubHypergraph [6]) Given a hypergraph $\mathcal{H} = (V, \mathcal{E})$, a reference vertex set $R \subseteq V$, a nonempty mandatory anchor set $A \subseteq R$, and a locality parameter $\epsilon \geq 0$, an ADSH is a vertex set $\widehat{S}_\epsilon$ that contains A and maximizes the parameterized R-subhypergraph density: $\widehat{S}_\epsilon \in \arg\max_{A \subseteq S \subseteq V} \rho_R^{\epsilon}(S)$.

The ADSH formulation requires the user to specify the locality parameter ϵ. Different choices impose different penalties

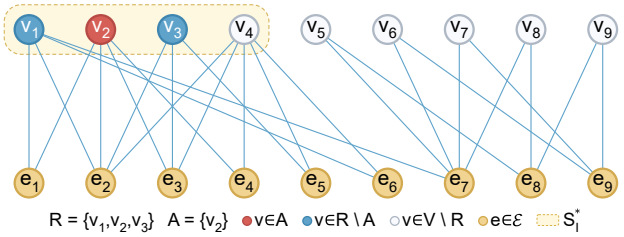


Fig. 1. An illustrative example of the I-ADSH problem.

on vertices outside R and can therefore produce different sub-hypergraphs. To provide fixed locality semantics and expose a discrete structure that supports efficient computation and maintenance, we introduce integer R-subhypergraph density and formulate the I-ADSH problem.

Definition 3: (Integer R-Subhypergraph Density) Given a hypergraph $\mathcal{H} = (V, \mathcal{E})$ and a reference vertex set $R \subseteq V$, the *unnormalized* R-*score* of a nonempty vertex set $S \subseteq V$ is $N_R(S) = 2m(S) - \mathrm{Vol}(S \setminus R)$. It is the two-unit reward of the internal hyperedges minus the incidence penalty of the selected vertices outside R. We use $\rho_R(S) = N_R(S)/|S| = 2\rho_R^1(S)$ for the unrounded density in this two-unit scale. The integer R-subhypergraph density of S is defined as

$$\bar{\rho}_R(S) = \lceil \rho_R(S) \rceil = \left\lceil \frac{2m(S) - \mathrm{Vol}(S \setminus R)}{|S|} \right\rceil. \tag{2}$$

Definition 4: (Integer Anchored Densest SubHypergraph) Given a hypergraph $\mathcal{H} = (V, \mathcal{E})$, a reference vertex set $R \subseteq V$, and a nonempty mandatory anchor set $A \subseteq R$, an I-ADSH is a vertex set S_I^* containing A that maximizes integer R-subhypergraph density: $S_I^* \in \arg\max_{S \supseteq A} \bar{\rho}_R(S)$.

We establish the relationships between I-ADSH, graph AADS, and hypergraph ADSH as follows.

Fact 1: For every nonempty $S \subseteq V$, $\bar{\rho}_R(S) = \lceil 2\rho_R^1(S) \rceil$. For an ADSH solution $\widehat{S}_1$ with $\epsilon = 1$ and an I-ADSH solution S_I^* under the same $(\mathcal{H}, R, A)$, $\bar{\rho}_R(S_I^*) = \lceil 2\rho_R^1(\widehat{S}_1) \rceil$. When $r = 2$, I-ADSH coincides exactly with graph AADS [13].

Proof: Setting $\epsilon = 1$ in Eq. (1) gives $2\rho_R^1(S) = \rho_R(S)$, and taking the ceiling proves the first equality. Since the feasible family is finite and the ceiling function is monotone, $\max_{S \supseteq A} \bar{\rho}_R(S) = \lceil 2 \max_{S \supseteq A} \rho_R^1(S) \rceil$, which proves the second equality. If $r = 2$, every hyperedge is a graph edge, so $m(S) = |E[S]|$ and the incident-edge count $d(v)$ equals graph degree; Eq. (2) is then exactly the integer R-subgraph density, and the two optimization problems coincide. □

Problem formulation. We study the computation and maintenance of I-ADSH solutions. Let ρ^* denote the optimum unrounded density and let $k^* = \lceil \rho^* \rceil$. For the *static problem*, given a hypergraph $\mathcal{H}$ and query (R, A), we seek the optimum integer level k^* together with an anchored set S_I^* attaining it. The returned set must maximize total excess at the lower integer boundary:

$$\beta = k^* - 1, \qquad F_\beta(S) = N_R(S) - \beta|S|, \\ S_I^* \in \arg\max_{S \supseteq A} F_\beta(S). \tag{3}$$

For the *dynamic problem*, V, R, and A remain fixed while previously absent hyperedges are inserted or existing hyperedges are deleted. After each update, we maintain the current optimum integer level k^* and an anchored set S_I^* satisfying Eq. (3) on the updated hypergraph.

The next theorem establishes the correctness guarantees of this output rule for both the static and dynamic problems.

*Theorem 1 (*Maximum-excess output guarantee*):* The optimum integer density satisfies $k^* \geq 0$. For $\beta = k^* - 1$, including $\beta = -1$ when $k^* = 0$, every maximizer T in Eq. (3) has integer density k^* and contains every set attaining ρ^*. Moreover, $0 \leq \rho^* - \rho_R(T) < 1$.

Proof: Since R is feasible and $N_R(R) = 2m(R) \geq 0$, we have $k^* \geq 0$. The complete-hyperedge indicator $\mathbf{1}[e \subseteq S]$ is supermodular, and the non-reference incidence penalty is modular. Thus N_R and F_β are supermodular. For an arbitrary real-valued optimum S^*, write $I = S^* \cap T$ and $U = S^* \cup T$. Both contain A, and $N_R(I) \leq \rho^*|I|$. Consequently, $F_\beta(U) - F_\beta(T) \geq F_\beta(S^*) - F_\beta(I) \geq (\rho^* - \beta)(|S^*| - |I|)$. Since $\beta = \lceil \rho^* \rceil - 1 < \rho^*$, omitting any vertex of S^* would strictly improve T, contradicting maximality. Hence $S^* \subseteq T$. Also $F_\beta(T) \geq F_\beta(S^*) = (\rho^* - \beta)|S^*| > 0$, so $k^* - 1 < \rho_R(T) \leq \rho^* \leq k^*$, proving both remaining claims. □

Example 1: Consider the hypergraph $H = (V, \mathcal{E})$ in Fig. 1, with reference set $R = \{v_1, v_2, v_3\}$ and mandatory anchor set $A = \{v_2\}$. The dashed region highlights $S_I^* = \{v_1, v_2, v_3, v_4\}$, which induces six hyperedges and incurs a volume penalty of $d_H(v_4) = 5$, giving $\bar{\rho}_R(S_I^*) = \lceil (2 \times 6 - 5)/4 \rceil = 2$. Checking all feasible sets containing A confirms that every other set has integer density at most 1. Thus, S_I^* is the unique optimal I-ADSH solution, with $k^* = 2$. At $\beta = k^* - 1 = 1$, this set also uniquely maximizes total excess, with $F_1(S_I^*) = 7 - 4 = 3$. Thus, this set satisfies the output requirement in Eq. (3).

III. Exact Static Algorithms

This section develops exact algorithms for static I-ADSH queries. We begin by formulating the density of an integer R-subhypergraph as a capacitated assignment of contributions from hyperedges. This formulation characterizes feasibility at a fixed density threshold through an anchor-bank orientation. We then optimize this orientation directly with Ceil-Balance, avoiding repeated threshold tests. This framework gives rise to two algorithms: CB-Global explicitly constructs the complete query-specific orientation, whereas CB-Local materializes the required orientation entries on demand through a shared index.

A. Exact Threshold Testing

Fix an integer threshold $\alpha \geq 1$. Instead of maximizing $\bar{\rho}_R(S)$ directly, we first test whether there exists an anchored set $S \supseteq A$ such that $\bar{\rho}_R(S) \geq \alpha$. Define the *$R$-incidence penalty* by $p_R(v) = 0$ for $v \in R$ and $p_R(v) = d(v)$ otherwise. This penalty is exactly the contribution incurred by selecting v, and therefore $\mathrm{Vol}(S \setminus R) = \sum_{v \in S} p_R(v)$. Because the unnormalized R-score $N_R(S)$ is integral, the threshold condition can be written as

$$\bar{\rho}_R(S) \geq \alpha \iff 2m(S) > \sum_{v \in S} \big(p_R(v) + \alpha - 1\big). \tag{4}$$

Eq. (4) separates the two units contributed by each internal hyperedge from a budget associated with each selected vertex. This observation suggests assigning two tokens from every hyperedge to vertex owners. Mandatory anchors require one additional mechanism: because every candidate contains A, the aggregate budget $|A|(\alpha-1)$ must remain available even for a hyperedge that is not incident to any anchor. We represent this shared capacity by a single anchor bank.

Definition 5: (Anchor-Bank Orientation) Given $(\mathcal{H}, R, A)$, an anchor-bank orientation comprises three components: *(i) Owners*, with an ordinary owner o_v for every $v \notin A$ and a shared anchor bank $a_\star$; *(ii) Token assignment*, where every hyperedge e supplies two tokens with allowed owners $P_A(e) = \{a_\star\} \cup \{o_v : v \in e \setminus A\}$ and integral assignment $x_{e,u} \geq 0$ satisfying $\sum_{u \in P_A(e)} x_{e,u} = 2$ for every e; and *(iii) Owner load*, defined by $L_x(u) = \sum_e x_{e,u}$ for each owner u.

The orientation records only where the tokens are stored and is independent of α. The tested threshold determines the number of tokens that each owner can accommodate.

Definition 6: (Level-α Feasibility) For an integer threshold $\alpha \geq 1$, let $\theta = \alpha - 1$. The budget of an ordinary owner is $B_\theta(o_v) = p_R(v) + \theta$, and the aggregate budget of the anchor bank is $B_\theta(a_\star) = |A|\theta$. An anchor-bank orientation x is *feasible at level* α if $L_x(u) \leq B_\theta(u)$ for every owner u.

Here, $p_R(v)$ accounts for the fixed incidence penalty of an ordinary owner, whereas θ is its per-vertex threshold budget. The anchor bank pools the corresponding threshold budgets of all mandatory anchors. No incidence penalty is required for the bank because $A \subseteq R$.

The feasibility condition is a capacitated assignment problem and admits a direct flow formulation. For a fixed α, let $\mathcal{N}_\alpha$ be the directed network with four components: *(i) Nodes*, consisting of a source s, a sink t, a hyperedge node h_e for every $e \in \mathcal{E}$, and one node for every owner u in Def. 5; *(ii) Supply arcs*, where each arc (s, h_e) for $e \in \mathcal{E}$ has capacity two, corresponding to the two tokens supplied by e; *(iii) Assignment arcs*, where each arc (h_e, u) for $e \in \mathcal{E}$ and $u \in P_A(e)$ has infinite capacity and allows the tokens of e to be assigned to u; and *(iv) Budget arcs*, where each arc (u, t) has capacity $B_\theta(u)$ and enforces the level-α budget of owner u.

We refer to the resulting maximum-flow computation as the *complete fixed-level flow test*. Saturating all supply arcs is equivalent to assigning every token within the available owner budgets. The following theorem shows that this condition is also equivalent to the absence of an anchored subhypergraph whose density reaches the tested threshold.

*Theorem 2 (*Anchor-bank threshold characterization*):* For $\alpha \geq 1$, the following are equivalent: *(i)* no $S \supseteq A$ satisfies $\bar{\rho}_R(S) \geq \alpha$; *(ii)* a level-$\alpha$-feasible anchor-bank orientation exists; *(iii)* the complete fixed-level flow test has value $2|\mathcal{E}|$. If the maximum-flow value is smaller than $2|\mathcal{E}|$, a minimum cut yields a set $S \supseteq A$ satisfying $\bar{\rho}_R(S) \geq \alpha$.

Proof: Statements *(ii)* and *(iii)* are equivalent because capacities are integral and a flow of value $2|\mathcal{E}|$ assigns both tokens of every hyperedge. It remains to relate this assignment to statement *(i)*. Suppose statement *(i)* holds, and consider any nonempty $F \subseteq \mathcal{E}$; the Hall condition for $F = \emptyset$ is immediate. Let S_F consist of A and all non-anchor endpoints of the hyperedges in F. The total capacity of the owner neighborhood of F is $\sum_{v \in S_F}(p_R(v) + \alpha - 1)$. By Eq. (4), this quantity is at least $2m(S_F) \geq 2|F|$. The capacitated Hall condition therefore holds for every F; hence the orientation is feasible.

Conversely, suppose some $S \supseteq A$ satisfies $\bar{\rho}_R(S) \geq \alpha$, and set $F = \mathcal{E}[S]$. Eq. (4) states that the $2|F|$ supplied tokens exceed the combined budget of the bank and the ordinary owners induced by S. The owner neighborhood of F is contained in this collection, so the Hall condition fails and no feasible orientation exists. Finally, consider a minimum cut of a deficient flow. Every allowed owner of a source-side hyperedge must also lie on the source side, as assignment arcs have infinite capacity. Those hyperedges are therefore internal to the set formed by A and the source-side ordinary owners. Their supply exceeds the set's total budget, and Eq. (4) gives the claimed anchored subhypergraph. □

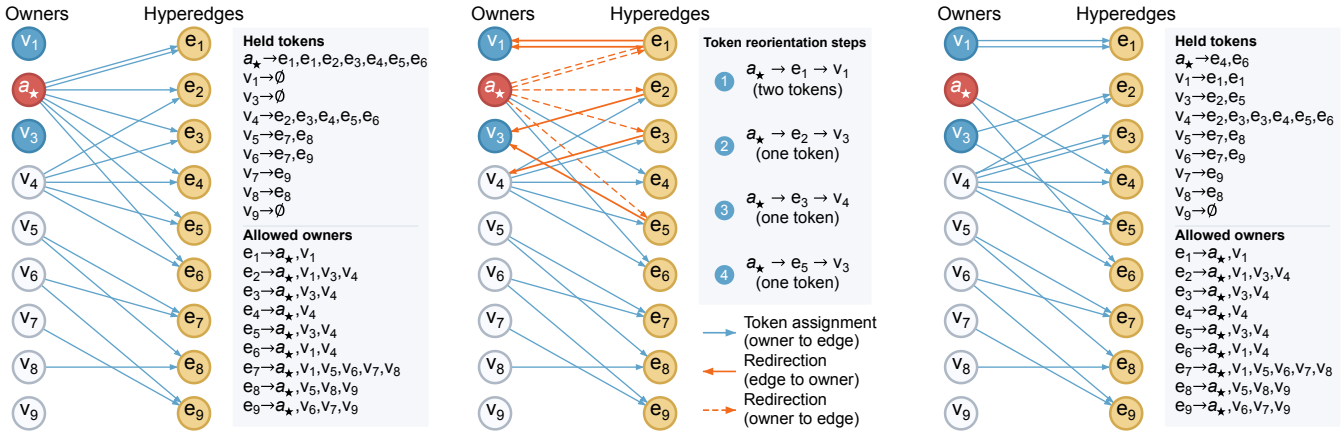
The flow construction and Thm. 2 together give an exact fixed-threshold test, which we call Full-Flow. Given $\mathcal{H}$, R, A, and an integer threshold α, Full-Flow constructs $\mathcal{N}_\alpha$ and computes a maximum flow. If the flow value is smaller than $2|\mathcal{E}|$, it returns (TRUE, S), where a minimum cut yields $S \supseteq A$ with $\bar{\rho}_R(S) \geq \alpha$. If all supply arcs are saturated, it returns $(\text{FALSE}, \perp)$, indicating that no such anchored set exists. Thus, the Boolean output directly indicates whether an anchored subhypergraph reaches the tested level. One call to Full-Flow uses a network with $O(|V|+|\mathcal{E}|)$ nodes and $O(|V|+|\mathcal{E}|+P)$ arcs, where $P = \sum_{e \in \mathcal{E}} |e|$ is the total number of pins.

We next use this characterization to optimize the orientation directly, avoiding a sequence of fixed-level flow tests.

B. Ceil-Balance *algorithms*

The fixed-level test changes only the owner budgets as α varies; the set of valid token assignments remains unchanged. This suggests using the orientation itself as the optimization state. To make the anchor bank comparable to an ordinary owner, we split it into $|A|$ interchangeable slots $a_1, \ldots, a_{|A|}$, each available to every hyperedge. The aggregate bank load is at most $|A|(\alpha-1)$ if and only if its tokens can be distributed so that every slot holds at most $\alpha - 1$ tokens. This transformation therefore preserves level feasibility while replacing one aggregate constraint with uniform per-owner constraints. In the resulting representation, the allowed owners of e are $P_A^{\text{sp}}(e) = \{a_1, \ldots, a_{|A|}\} \cup \{o_v : v \in e \setminus A\}$.

For an orientation x in the split-bank representation, define the *adjusted loads* as $\lambda_x(o_v) = L_x(o_v) - p_R(v)$ for $v \notin A$ and $\lambda_x(a_i) = L_x(a_i)$ for $i = 1, \ldots, |A|$, and let $U(x) = \max_u \lambda_x(u)$. Subtracting $p_R(v)$ absorbs the fixed non-reference penalty into the load of an ordinary owner; bank slots require no such offset because $A \subseteq R$. Consequently, an orientation x is feasible at level α if and only if $U(x) \leq \alpha - 1$. Because k^* is the optimum I-ADSH level, the threshold characterization implies that $k^* \leq U(x)$ for every orientation x. Thus, minimizing $U(x)$ yields a tighter upper bound on k^*.



(a) Initial Orientation (b) Token Reorientation Process (c) Final Orientation

Note. Blue arcs show owner-to-hyperedge token assignments, one arc per token. Gray *Allowed owners* panels specify the possible hyperedge-to-owner residual transitions; only those used in the displayed paths are drawn. Repeated labels in *Held tokens*, such as e_1, e_1, denote two tokens of the same hyperedge.

Fig. 2. Orientation balancing process of the CB-Global algorithm.

To reduce $U(x)$ while preserving the token-assignment constraints, we move between orientations through a *residual assignment graph*. The graph contains an arc $u \to h_e$ for each token of e currently held by owner u, and an arc $h_e \to u'$ for every alternative owner allowed by e in the split-bank representation. Reversing an alternating path from u to u' transfers one token between its endpoints while preserving all intermediate loads and the two-token constraint of every hyperedge. In particular, a path from an owner with maximum adjusted load to an owner with adjusted load at most $U(x)-2$ defines a valid balancing move. The following theorem establishes the optimality of this residual balancing process and characterizes its stopping condition.

*Theorem 3 (*Min–max balance characterization*):* Let x range over all complete two-token orientations in the split-bank representation. Then the optimum integer density satisfies $k^* = \min_x U(x)$. For any orientation with $U(x) \geq 1$, consider a residual search from an owner of adjusted load $U(x)$. If the search exhausts the reachable region without finding an owner with adjusted load at most $U(x)-2$, the reached owners induce a set $S \supseteq A$ satisfying $\bar{\rho}_R(S) \geq U(x)$.

Proof: For threshold α, an orientation is feasible exactly when $U(x) \leq \alpha - 1$. By Thm. 2, such an orientation exists if and only if no $S \supseteq A$ has $\bar{\rho}_R(S) \geq \alpha$. Hence $1+\min_x U(x)$ is the smallest excluded integer level, and the largest attainable level is $\min_x U(x)$.

For the second claim, write $U = U(x)$, and let C and F denote the reached owners and hyperedges. Since $U \geq 1$, the root holds a token, so the search reaches a hyperedge and then every bank slot. Residual closure has two useful consequences: every token held in C belongs to an edge in F, and every allowed owner of an edge in F lies in C. Thus $\sum_{u \in C} L_x(u) = 2|F|$. Moreover, every reached owner has adjusted load at least $U-1$, while the root has adjusted load U. The token supply of F therefore strictly exceeds the total level-U budget of C.

Set $S = A \cup \{v : o_v \in C\}$. All edges in F are internal to S, and C contains exactly the ordinary owners induced by S together with all bank slots. It follows that $2m(S) > \sum_{v \in S}(p_R(v) + U - 1)$. By Eq. (4), $\bar{\rho}_R(S) \geq U$. □

Thm. 3 reduces I-ADSH to balancing adjusted loads. We therefore seek an inexpensive initial orientation constructible for different queries. Fix a deterministic vertex order. For each hyperedge e, assign one token to each of the first $\min\{2, |e \setminus R|\}$ non-reference vertices, and distribute any remaining tokens among the bank slots in round-robin order. We call the resulting *canonical orientation* x^0. Every ordinary owner has nonpositive adjusted load: each non-reference vertex receives at most one token from each incident hyperedge, so $L_{x^0}(o_v) \leq p_R(v)$, while owners in $R \setminus A$ receive no tokens. Consequently, all positive adjusted load is concentrated in the bank, whose tokens are distributed evenly among its slots.

Starting from x^0, Ceil-Balance first certifies the optimum integer level and then constructs the maximum-excess answer. It maintains the lower bound $L = \bar{\rho}_R(R) = \lceil 2m(R)/|R| \rceil$ and the upper bound $U(x)$. While the optimum remains uncertified and $U(x) > L$, the algorithm starts a search from an owner with maximum adjusted load and looks for a target whose adjusted load is at most $U(x) - 2$. A residual path transfers tokens between its endpoints. An exhaustive search that finds no target certifies $k = U(x)$ by Thm. 3; reaching $U(x) = L$ certifies $k = L$. The variable k records the certified optimum, so $k = k^*$ once certification completes.

After the level k has been certified, the algorithm processes every unclosed owner with adjusted load k, using targets whose adjusted load is at most $k - 2$. A failed search adds its reached region to a retained closed set C, which subsequent searches skip. Processing terminates only when no unclosed owner has adjusted load k. The returned vertex set is $S_I^* = A \cup \{v : o_v \in C\}$. The same closure-collection step is required when the lower and upper bounds coincide before any search is performed.

A closure certificate for total excess. For a complete split-bank orientation, define $D_\beta(x) = \sum_u \max\{0, \lambda_x(u) - \beta\}$. Let $O(S)$ consist of the ordinary owners of $S \setminus A$ and all bank slots. Every token of an internal edge of S is held by an owner in $O(S)$. Therefore,

$$F_\beta(S) \le \sum_{u \in O(S)} (\lambda_x(u) - \beta) \le D_\beta(x). \tag{5}$$

This inequality holds for every integer β, including $\beta = -1$.

Lemma 1: Suppose that C is residual-closed, contains all bank slots and every owner with $\lambda_x(u) > \beta$, and satisfies $\lambda_x(u) \ge \beta$ for every $u \in C$. Then $S_C = A \cup \{v : o_v \in C\}$ maximizes F_β.

Proof: An edge with a token held in C has all its allowed owners in C, so it is internal to S_C. Conversely, every internal edge has all its allowed owners in C, and both its tokens are held there. Therefore $2m(S_C) = \sum_{u \in C} L_x(u)$. Subtracting penalties and the boundary cost gives

$$F_\beta(S_C) = \sum_{u \in C} (\lambda_x(u) - \beta) = D_\beta(x).$$

The last equality follows because C includes every positive-excess owner and no negative-excess owner. Eq. (5) proves optimality. □

Building on the Ceil-Balance framework, we next present two algorithms, CB-Global and CB-Local, in detail.

Algorithm 1: CB-Global$(\mathcal{H}, R, A)$

Input : Hypergraph $\mathcal{H}$, reference set R, nonempty anchors $A \subseteq R$
Output: Optimum level k and a maximum-excess I-ADSH S_I^*

Initialize $x \leftarrow x^0$, loads, and held-edge lists;
$L \leftarrow \bar{\rho}_R(R)$; $k \leftarrow \perp$; $C \leftarrow \emptyset$;
loop
  $U \leftarrow \max_{u \notin C} \lambda_x(u)$;
  if $k = \perp$ **and** $U \le L$ **then** $k \leftarrow L$;
  if $k = 0$ ***and*** *zero roots have not been discovered* **then**
    Discover all ordinary owners with $\lambda_x(u) = 0$;
    Add every ordinary owner with $\lambda_x(o_v) = L_x(o_v) = 0$, and every bank slot, to C;
    Retain all other zero roots for search; **continue**;
  if $k \ne \perp$ ***and*** $U < k$ **then**
    $S_I^* \leftarrow A \cup \{v : o_v \in C\}$;
    return (k, S_I^*);
  Choose $r \notin C$ with $\lambda_x(r) = U$;
  $(C_r, T, \mathrm{par}) \leftarrow$ BatchedBFS$(x, r, U; C)$;
  if $T = \emptyset$ **then**
    if $k = \perp$ **then** $k \leftarrow U$;
    $C \leftarrow C \cup C_r$;
  else
    Recover the r-to-t path π_t from par for each $t \in T$;
    Sort T by nondecreasing adjusted load;
    for *each* $t \in T$ *whose path* π_t *remains residual and satisfies* $\lambda_x(r) - \lambda_x(t) \ge 2$ **do**
      $\delta \leftarrow \min\{\lfloor(\lambda_x(r) - \lambda_x(t))/2\rfloor, \min_{(u,e,u') \in \pi_t} x_{e,u}\}$;
      Transfer δ tokens along π_t; update assignments, loads, held-edge lists, and maximum keys;

CB-Global **algorithm.** Alg. 1 stores every token assignment, owner load, and held-edge list. Its batched Breadth-First Search (BFS) alternates between an owner, its held edges, and the allowed owners of those edges. It records parent paths and collects targets in nondecreasing order of adjusted load for subsequent transfers. The search may stop after collecting a prescribed positive number of targets; it returns an empty target set only after the search queue is exhausted. When previously closed regions are skipped, their union with the owners reached by a failed search remains residual-closed.

In Alg. 1, $U = \max_{u \notin C} \lambda_x(u)$ is the maximum adjusted load among unclosed owners. It equals $U(x)$ while $C = \emptyset$, but may fall below the certified level k during closure collection even though $U(x) \ge k$. The maximum over an empty candidate set is $-\infty$. Once k is certified, every unclosed root has adjusted load k, and every searched non-target owner has adjusted load at least $k - 1$. Failed regions remain unchanged because subsequent transfers avoid them. At $k = 0$, ordinary owners of adjusted load zero must be discovered globally: they need not have appeared in a positive-load heap. A zero-load root with no held token is an isolated residual owner and can be added directly to the answer. Every bank slot then has zero load and is also included.

Example 2: Fig. 2 illustrates CB-Global on the instance of Fig. 1, with $a_\star$ representing v_2. The initial maximum adjusted load is 7 and the lower bound is $L = \lceil 2/3 \rceil = 1$. The first BFS batch transfers two tokens along $a_\star \to e_1 \to v_1$ and one along $a_\star \to e_2 \to v_3$, reducing the maximum adjusted load to 4. The next batches transfer one token along $a_\star \to e_3 \to v_4$ and one along $a_\star \to e_5 \to v_3$, reducing the maximum adjusted load to 3 and then 2. The fourth BFS from $a_\star$ finds no target of adjusted load at most zero and reaches $C = \{a_\star, v_1, v_3, v_4\}$, certifying $k = 2$. It contains all three owners of adjusted load 2: $a_\star$, v_1, and v_3. Thus all positive-excess roots at $\beta = 1$ have been covered, and no further transfer is needed. The returned set is $S_I^* = \{v_1, v_2, v_3, v_4\}$, with $F_1(S_I^*) = 7 - 4 = 3 = D_1(x)$. All token assignments in Fig. 2(c) are retained.

Theorem 4: Alg. 1 returns the optimum integer level and a maximum-excess answer satisfying Thm. 1. Let Ξ_G count residual inspections, transfers, root and heap operations, closed-region maintenance, and answer extraction, and let T_G be the peak additional records retained. Its time is $O(|V| + |\mathcal{E}| + P + \Xi_G)$ and its space is $O(|V| + |\mathcal{E}| + P + T_G)$. The total transferred token units are at most $\Phi(x^0)/2$, where $\Phi(x) = \sum_u \lambda_x(u)^2$ and $\Phi(x^0) \le 4|\mathcal{E}|^2 + P^2$. The number of search batches is $O(1 + |V| + |A| + |\mathcal{E}|^2 + P^2)$.

Proof: Thm. 3 and the lower bound certify k. Every transfer from loads h, z with $d = h - z \ge 2$ uses an integer $1 \le \delta \le \lfloor d/2 \rfloor$ and changes the potential by $-2\delta(d - \delta) \le -2\delta$. It preserves the complete assignment and all intermediate loads. Every successful batch transfers at least one token; invalidated later parent paths are skipped. After certification no adjusted load exceeds k. A failed search adds at least one previously unclosed root to C, and its non-target owners have adjusted load at least $k - 1$. The union remains closed, and later paths cannot alter it. Termination therefore gives a closed C containing all adjusted-load-k owners. For $k \ge 1$, the first failed search reaches an edge and all bank slots.

Algorithm 2: CB-Local$(\mathcal{H}, \mathcal{I}_{\mathcal{H}}, R, A)$

Input : Hypergraph $\mathcal{H}$, shared index $\mathcal{I}_{\mathcal{H}}$, query R, A
Output: Optimum level k and a maximum-excess I-ADSH S_I^*

Prepare sparse corrections representing x^0 from $\mathcal{I}_{\mathcal{H}}$ and R, A;
$L \leftarrow \bar{\rho}_R(R)$; $k \leftarrow \perp$; $C \leftarrow \emptyset$;
Initialize a lazy maximum heap with the bank slots;
Execute the loop of Alg. 1, using:
    LazyBatchedBFS for implicit search;
    sparse assignment and load corrections for transfers;
    sparse closed-owner and answer records for C;
    indexed loads for complete zero-root discovery if $k = 0$;
return (k, S_I^*);

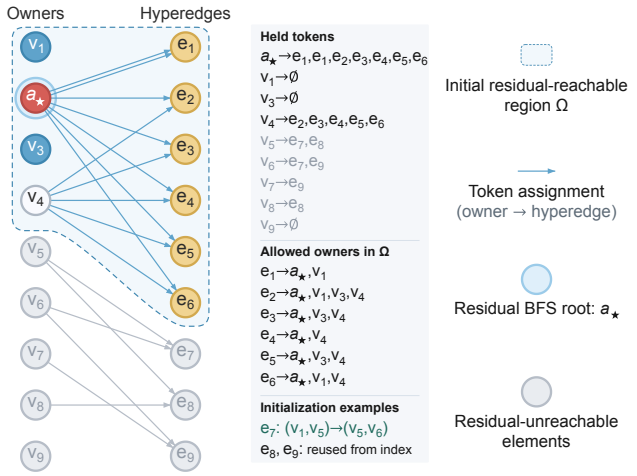
For $k = 0$, root discovery includes all zero-load bank slots and all untracked zero roots. Lem. 1 with $\beta = k - 1$ proves maximum excess, and Thm. 1 proves the output guarantee.

Initially every ordinary adjusted load lies between $-d(v)$ and zero, and bank slots hold at most $2|\mathcal{E}|$ tokens in total. Thus $\Phi(x^0) \le 4|\mathcal{E}|^2 + \sum_{v \notin R} d(v)^2 \le 4|\mathcal{E}|^2 + P^2$. Successful batches are bounded by potential decrease; unsuccessful batches close a new owner and occur at most $|V|$ times. Construction uses $O(|V| + |\mathcal{E}| + P)$ time and space. All subsequent work is counted in Ξ_G, including root discovery, repeated record inspection, and closed-set construction. Held-edge and heap records may outlive their validity; their peak retained storage is included in T_G. □

CB-Local algorithm. Alg. 2 presents CB-Local, which follows the same balancing rule as CB-Global but represents the query-specific orientation implicitly. It retrieves token assignments and owner loads on demand during residual searches, while sharing unchanged base state across queries.

A query-independent index $\mathcal{I}_{\mathcal{H}}$ supports these operations by storing hypergraph incidences, base token assignments, owner loads, and held-edge lists. The base orientation assigns each hyperedge's two tokens to its first two pins under the fixed vertex order, independently of R and A. The held-edge lists allow searches to enumerate an owner's residual connections without scanning all hyperedges. To obtain x^0, only hyperedges with a base token assigned to a vertex in R require reassignment. These hyperedges are collected from the indexed held-edge lists of vertices in R and deduplicated. Query-specific changes are recorded as sparse assignment overrides, load offsets relative to the base values, and additional held-edge entries. Overrides and load offsets are updated in place; obsolete held-edge entries are filtered during search. Together, the shared index and these corrections provide on-demand access to the complete orientation.

To select the next search root, CB-Local maintains a maximum heap keyed by adjusted load. The heap initially contains only the bank slots, since all ordinary owners have nonpositive adjusted loads under x^0. Each path transfer changes only its endpoint adjusted loads, and the updated endpoint keys are inserted into the heap. Stale entries and entries for closed owners are discarded during root selection. The heap thus identifies the maximum positive adjusted load among unclosed owners during balancing and covers all remaining roots with adjusted load k when $k > 0$. When $k = 0$, the algorithm scans all ordinary owners once to discover untracked roots with adjusted load zero; those holding no tokens are included directly in the answer.



Note. Token assignments, repeated labels, and the *Allowed owners* panel follow the conventions in Fig. 2.

Fig. 3. Initial orientation and residual reachability of the CB-Local algorithm.

In Alg. 2, LazyBatchedBFS performs residual searches on the implicit orientation. It retrieves token assignments from sparse overrides when available or the shared index otherwise, and computes owner loads from indexed base loads and recorded offsets. For each reached owner, it enumerates indexed and appended held-edge entries, checking current ownership to filter obsolete entries. It skips the retained closed set C and follows the same target-selection and batching rules as BatchedBFS. An empty target set is returned only after exhausting the search queue. Thus, the implicit and explicit representations support the same level certification and complete-root collection, preserving the maximum-excess output guarantee.

Example 3: Fig. 3 shows CB-Local using the same R and A in Fig. 1. The canonical orientation x^0 is represented implicitly by a query-independent shared index and sparse query-specific corrections. For example, during query initialization, CB-Local records an override for e_7, changing its owners from (v_1, v_5) to (v_5, v_6) because $v_1 \in R$. The assignments of e_8 and e_9 remain in the shared index without being copied into the query state. The dashed boundary marks the initial residual-reachable region Ω from $a_\star$, containing owners $\{a_\star, v_1, v_3, v_4\}$ and hyperedges $\{e_1, e_2, e_3, e_4, e_5, e_6\}$. The gray elements lie outside Ω and are not reached by residual search. For this instance, CB-Local performs the same token transfers and complete-root collection as CB-Global in Fig. 2. The closure contains every owner of adjusted load 2, so the maximum-excess answer remains $(2, S_I^*)$. The gray elements are outside the residual BFS region; query initialization still inspects e_7 and its pins to construct its correction.

Theorem 5: Alg. 2 returns the optimum integer level and a maximum-excess answer satisfying Thm. 1. Its shared index

takes $O(|V| + |\mathcal{E}| + P)$ preprocessing time and space. Let $P_R = \sum_{e:e\cap R\neq\emptyset} |e|$ and $Q_R = |R| + P_R$. With expected constant-time hash operations, a query takes $O\big(Q_R \log(2 + Q_R) + \mathbf{1}[k = 0]|V| + \Xi_L\big)$ expected time and $O(Q_R + |A| + T_L)$ additional space. Here Ξ_L includes all searches, transfers, heap operations, closure collection, and answer construction. T_L counts peak retained search, override, held-edge, heap, closed-owner, and answer records.

Proof: The index plus query corrections represents every assignment and exact owner load of x^0. Transfers maintain these values and append every new held edge; filtering an obsolete record cannot remove a current residual connection. The implicit search therefore follows the same valid residual paths and closed-region rules as the explicit representation. The heap contains every owner of positive adjusted load, since untracked initial ordinary adjusted loads are nonpositive and every changed endpoint is inserted. At a positive optimum this also covers every positive-excess root. At zero optimum the explicit discovery scan covers the missing zero roots. Thm. 4 and Lem. 1 establish correctness and termination.

Each pin and token contributes a constant number of index records. Preparing R, A, deduplicating affected edge identifiers, and computing $m(R)$ take $O(Q_R \log(2 + Q_R))$ expected time and $O(Q_R + |A|)$ temporary space. The zero-root scan is charged separately. All remaining work is charged to Ξ_L, including processing every boundary root and extracting the complete answer. Counting retained records, including obsolete heap and held-edge entries, gives the T_L space term. Residual work can cover the entire hypergraph in the worst case. □

IV. Exact Dynamic Maintenance

In this section, we develop an exact dynamic algorithm for maintaining an I-ADSH under hyperedge insertions and deletions, with the vertex set V, reference set R, and anchor set A fixed. We first specify the maintained state and its initialization, and establish that a single update changes the optimum integer level by at most one. We then present Dynamic-Path, which reuses the maintained orientation and repairs it along residual paths to recover the exact optimum and the corresponding maximum-excess answer after each update.

A. Maintenance framework and update bound

Maintained invariant. For each snapshot, we maintain the optimum level k, a set $S_I^* \in \arg\max_{S\supseteq A} F_{k-1}(S)$, its numerator $N_R(S_I^*)$, and a complete orientation x^{k+1} feasible at level $k + 1$. For an updated hypergraph $\mathcal{H}'$, let k' and $S_I^{*\prime}$ denote the new optimum level and answer, respectively. We use N'_R, ρ'_R, $\bar{\rho}'_R$, and $F'_\beta(S) = N'_R(S) - \beta|S|$ for the corresponding functions on $\mathcal{H}'$. In Alg. 4, x denotes the working orientation initialized from x^{k+1}. Thm. 1 provides the lower certificate and establishes containment of the optimal core, while the orientation rules out level $k + 1$. Each update must therefore restore both the exact optimum level and the maximum-excess property of the returned set.

Dynamic initialization. We initialize the state using CB-Global or CB-Local, retaining the maximum-excess set and the final orientation. For an implicit orientation, query overrides and indexed base assignments jointly determine the owner of every token. To support dynamic updates, we reconstruct owner loads and held-edge lists once and consolidate the split bank slots into $a_\star$. Since their budgets sum to the aggregate bank budget, this consolidation preserves feasibility at level $k+1$. We also retain or construct the complete set of boundary roots required for answer extraction.

One-level update bound. The next theorem gives a size-independent update bound, despite an update possibly changing the densities of many candidate sets.

Theorem 6: A single hyperedge update changes the optimum I-ADSH level by at most one.

Proof: Consider first the insertion of a hyperedge e. For any fixed $S \supseteq A$, let $q = |e \cap (S \setminus R)|$. The change in its unnormalized score is $\Delta N_R(S) = 2\mathbf{1}[e \subseteq S] - q$. If $e \nsubseteq S$, then $\Delta N_R(S) = -q$ and $|\Delta N_R(S)| \leq |S|$. If $e \subseteq S$, then $\Delta N_R(S) = 2 - q$; because $|S| \geq 2$ and $0 \leq q \leq |S|$, we again have $|\Delta N_R(S)| \leq |S|$. Therefore, $|\rho'_R(S) - \rho_R(S)| \leq 1$, and taking ceilings gives $|\bar{\rho}'_R(S) - \bar{\rho}_R(S)| \leq 1$ for every feasible set S. The feasible family is unchanged, so taking its maximum yields $|k' - k| \leq 1$. A deletion reverses the sign of the same score change, and the same argument applies. □

Thm. 6 restricts the new optimum to $\{k-1, k, k+1\}$. We determine the new level using the retained orientation, then restore the maximum-excess answer at its new boundary.

B. Efficient maintenance with path repair

Building on the one-level bound, Dynamic-Path repairs the maintained orientation x^{k+1} along residual paths to decide the candidate levels. Repair provides both outcomes of an exact threshold test: eliminating all overload rules out the tested level, whereas a failed search identifies a set attaining it. The same assignment therefore supports threshold decisions and provides the upper-bound orientation for subsequent updates.

Effect of an update. Let e be the changed hyperedge. Insertion assigns two new tokens to allowed owners and increases the budget of each pin in $e \setminus R$ by one. Only recipients of the new tokens can become overloaded. Deletion removes its two tokens of e but also decreases those non-reference budgets by one; it can therefore create overload despite reducing the token supply. In either case, the assignment remains complete, and every owner that can violate its unchanged-level budget belongs to $D = \{a_\star\} \cup \{o_v : v \in e \setminus A\}$. The maintained solution is updated using its membership array. Define $q_e(S) = 2\mathbf{1}[e \subseteq S] - |e \cap (S \setminus R)|$. Insertion adds $q_e(S_I^*)$ to the stored score $N_R(S_I^*)$, whereas deletion subtracts it, yielding the updated score $N'_R(S_I^*)$ for that vertex set. This avoids a hypergraph scan when checking the updated density.

Residual repair. Fix an integer level $\alpha \geq 1$ and let $\theta = \alpha - 1$. Write $\mathcal{O} = \{a_\star\} \cup \{o_v : v \in V \setminus A\}$ for all owners.

*Definition 7 (*Residual repair instance*):* For a complete orientation x, define the overload of owner u as $\mathrm{over}_x(u) =$

Algorithm 3: Path-Repair$(\mathcal{H}, R, A, x, \alpha, D)$

Input : Unweighted hypergraph $\mathcal{H} = (V, \mathcal{E})$, reference set $R \subseteq V$, nonempty anchors $A \subseteq R$, complete anchor-bank orientation x, tested integer level $\alpha \geq 1$, and candidate owners D containing every overloaded owner

Output: $(found, S, x)$: if $found =$ TRUE, then $\bar{\rho}_R(S) \geq \alpha$; otherwise $S = \perp$ and x is feasible at level α

while *some owner in* D *is overloaded* **do**
  $(C, \pi) \leftarrow$ Residual BFS from the overloaded owners in D;
  if $\pi = \perp$ **then** ▷ No repair path exists
    $S \leftarrow A \cup \{v : o_v \in C\}$;
    return (TRUE, S, x);
  Transfer δ tokens along π as in Def. 7, updating assignments, loads, and held-edge lists;
  Refresh the retained boundary-root memberships of both endpoints of π;
return (FALSE, $\perp, x$); ▷ x is feasible at α

Algorithm 4: Dynamic-Path$(\mathcal{H}, R, A, k, S_I^*, x, \Delta)$

Input : State (k, S_I^*, x^{k+1}), fixed query R, A, update Δ of hyperedge e

Output: Updated level k', maximum-excess answer $S_I^{*\prime}$, and upper orientation x

$\mathcal{H}' \leftarrow \Delta(\mathcal{H})$; $D \leftarrow P_A(e)$;
Update penalties, cached score, token assignments, and root records;
$(found, S, x) \leftarrow$ Path-Repair$(\mathcal{H}', R, A, x, k+1, D)$;
if *found* **then**
  $k' \leftarrow k+1$;
  $(found, S, x) \leftarrow$ Path-Repair$(\mathcal{H}', R, A, x, k+2, D)$;
else if $\bar{\rho}'_R(S_I^*) \geq k$ **or** $k = 0$ **then** $k' \leftarrow k$;
else
  $(found, S, x) \leftarrow$ Path-Repair$(\mathcal{H}', R, A, x, k, \mathcal{O})$;
  $k' \leftarrow k$ if *found*, and $k-1$ otherwise;
Refresh boundary roots at $\beta = k' - 1$, checking all owners if $k' \neq k$ and only owners affected by the update or repair otherwise;
$(S_I^{*\prime}, x) \leftarrow$ Excess-Closure$(\mathcal{H}', R, A, x, k')$;
Store the answer membership and $N'_R(S_I^{*\prime})$;
return $(\mathcal{H}', k', S_I^{*\prime}, x)$; ▷ x excludes $k'+1$

$\max\{0, L_x(u) - B_\theta(u)\}$ and its spare capacity as $\mathrm{def}_x(u) = \max\{0, B_\theta(u) - L_x(u)\}$. The residual assignment graph contains $u \to h_e$ with capacity $x_{e,u}$ when u holds a token of e, and $h_e \to u'$ with infinite capacity for each $u' \in P_A(e)$. For a residual path π from an overloaded owner r to a spare-capacity owner t, let $c_\pi = \min_{(u,h_e)\in\pi} x_{e,u}$ and $\delta = \min\{c_\pi, \mathrm{over}_x(r), \mathrm{def}_x(t)\}$. Transferring δ tokens along π reduces the load of r and increases that of t by δ, leaving internal owner loads unchanged.

Given candidates D containing every overloaded owner, Path-Repair (Alg. 3) repeatedly searches from overloaded owners for spare capacity and transfers tokens along a path. If no path exists, it returns the anchored set induced by the reached owners. As in Full-Flow, *found* refers to finding a set at the tested level, not to repairing the orientation: a feasible repair returns false. The residual graph is generated on demand from held-edge lists and hyperedge pins. Each held-edge entry is checked for edge activity and ownership, so obsolete records cannot create residual arcs. Search extent is determined by residual reachability rather than a predefined region.

Theorem 7: If D contains every overloaded owner, the procedure terminates with either a feasible level-α orientation or a set $S \supseteq A$ satisfying $\bar{\rho}_R(S) \geq \alpha$. It returns $found =$ FALSE if and only if a feasible orientation exists.

Proof: Each transfer preserves the two tokens of every hyperedge, decreases total overload by the positive integer δ, and creates no new overloaded owner. Thus D continues to contain all overloaded owners, and the procedure terminates. If none remain, the returned orientation is feasible.

Otherwise, the final BFS exhausts its queue without reaching spare capacity. Let C and F be the reached owners and hyperedges. Every hyperedge held by an owner in C is explored, and every allowed owner of an explored hyperedge is reached. Hence all tokens held in C come from F, and all tokens of F are held in C. Moreover, $a_\star \in C$: it is either a root or is reached through an edge held by an overloaded ordinary root, since the bank is an allowed owner of every hyperedge. Such an edge exists because an overloaded owner has positive load and all budgets are nonnegative. No reached owner has spare capacity, and at least one is overloaded. For $S = A \cup \{v : o_v \in C\}$, we therefore have

$$2|F| = \sum_{u\in C} L_x(u) > \sum_{u\in C} B_\theta(u) = \sum_{v\in S} (p_R(v) + \theta).$$

Every edge in F lies inside S. Eq. (4) then gives $\bar{\rho}_R(S) \geq \alpha$, which rules out a feasible orientation by Thm. 2. The two outcomes are thus exact and mutually exclusive. □

The first repair tests the unchanged level $k+1$. Because the stored orientation was feasible before the update, the number of successful path transfers depends on the introduced overload, as bounded below.

Lemma 2: Fix α and a feasible pre-update orientation. After inserting e and assigning its two new tokens to allowed owners, total overload is at most two. After deleting e, total overload is at most $|e \setminus R|$. Thus, the repair at this unchanged level performs at most two augmentations after insertion and at most $|e \setminus R|$ augmentations after deletion.

Proof: Insertion increases the penalty budget of each owner in $e \setminus R$ and decreases no budget, so only the two added tokens can create overload. Deletion removes the tokens of e but lowers by one the budget of each owner in $e \setminus R$; each such owner contributes at most one overload unit. Every integral transfer reduces total overload by at least one. □

Complete-root extraction via Excess-Closure**.** Once the new optimum k' is known, Excess-Closure applies the same residual transfer mechanism at boundary $\beta = k' - 1$, starting from *all* owners whose loads exceed B_β. It continues transferring to spare-capacity owners until no root has a residual path to spare capacity, then returns $S_I^{*\prime} = A \cup \{v : o_v \in C\}$, where C is the complete region reached from every remaining overloaded root. At $k' = 0$, budgets may be negative; overload is still $\max\{0, L_x(u) - B_{-1}(u)\}$ and spare capacity is its positive reverse. The negative-budget bank is a root even if

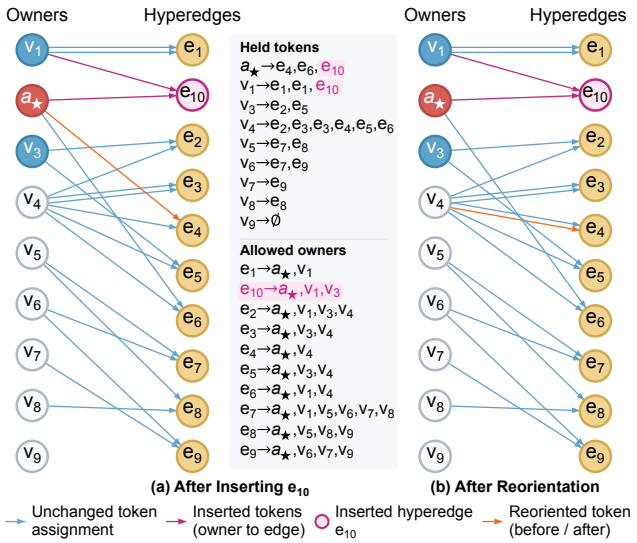


Fig. 4. Dynamic-Path for Hyperedge Insertion.

it holds no token. This signed-budget extraction maximizes excess without performing a level-zero feasibility test.

For aggregate-bank owners on $\mathcal{H}'$, define $D_\beta(x) = \sum_u \max\{0, L_x(u) - B_\beta(u)\}$. The same argument as Eq. (5) gives $F'_\beta(S) \le D_\beta(x)$ for every anchored S. The final multi-source search contains every remaining overloaded owner and reaches no owner with spare capacity. It contains the bank, either through a held edge when $k' > 0$, or as a negative-budget root when $k' = 0$. Its residual closure therefore attains this bound exactly, by the aggregate-bank version of Lem. 1.

The complete root set is retained between updates. Every token transfer in Path-Repair or Excess-Closure refreshes the boundary-root memberships of its endpoints; changes to non-reference penalties refresh the affected owners. If the level changes, all owners are checked against the new boundary; if it stays fixed, the maintained root set is reused. Every extraction still starts from all current overloaded roots. Constructing the output, updating membership records, and refreshing its stored numerator are included in maintenance work.

Unified dynamic maintenance. Alg. 4 first restores the level bracket using at most two Path-Repair calls. It then performs complete-root extraction at the decided level, even when the cached set already certifies that level. The upper-bound orientation is reused throughout.

The following example illustrates how Dynamic-Path restores the upper-bound certificate when an insertion raises the optimum on the hypergraph of Ex. 2. The case of hyperedge deletion is analogous, and we omit the corresponding example.

Example 4: Fig. 4 shows inserting $e_{10} = \{v_1, v_2, v_3\}$ into the hypergraph in Fig. 1, with R and A fixed. Before insertion, the optimal integer density is $k = 2$, and the maintained orientation in Fig. 2(c) is feasible at level $k+1 = 3$. Inserting e_{10} assigns its two tokens to $a_\star$ and v_1 (magenta arcs in Fig. 4(a)). Their adjusted loads rise from 2 to 3, exceeding the level-3 limit by one. Dynamic-Path starts residual BFSs from both overloaded owners and stops at the first owner with spare capacity. In the gray panel of Fig. 4(a), $a_\star$ holds an e_4 token, and v_4 is an allowed owner, yielding the path $a_\star \to e_4 \to v_4$. Since v_4 has adjusted load 1, transferring one token changes the adjusted loads of $a_\star$ and v_4 from $(3, 1)$ to $(2, 2)$, as shown in Fig. 4(b). The *Held tokens* list moves one e_4 token from $a_\star$ to v_4, while the *Allowed owners* list is unchanged. The second BFS from the remaining overloaded owner v_1 exhausts the reachable region without finding spare capacity. Replacing the bank $a_\star$ by its anchor v_2 in the reached owner set yields $S' = \{v_1, v_2, v_3, v_4\}$. With seven induced hyperedges and volume penalty 5, S' has $\bar{\rho}'_R(S') = \lceil (2 \times 7 - 5)/4 \rceil = 3$. Thm. 6 bounds the new optimum by $k + 1 = 3$, establishing the optimality of S'. At boundary $\beta = 2$, the only overloaded owner is v_1. Its complete residual closure is $\{a_\star, v_1, v_3, v_4\}$; every reached owner meets or exceeds its boundary budget. Thus $F'_2(S') = 9 - 2 \times 4 = 1$ is the maximum excess. No additional transfer is needed for complete-root extraction. Dynamic-Path returns $(3, S^{*\prime}_I)$ with $S^{*\prime}_I = S'$ and retains the level-4 feasible orientation in Fig. 4(b).

Theorem 8: Alg. 4 restores the maintained invariant after each update. It uses at most two level-decision repair calls and one complete-root extraction, with no materialized maximum-flow computation. Moreover, let $n = |V|$, $P = \sum_{f \in \mathcal{E}'} |f|$, and let H be the peak number of retained hyperedge and held-edge records during an update. Starting from a valid maintained state, Dynamic-Path performs $O(n)$ residual searches. With multi-source BFS and constant-time record operations, its total update time is $O(n(n + P + H))$.

Proof: The update can introduce overload at level $k+1$ only in $D = P_A(e)$. Path transfers create no new overloaded owner, so D is sufficient for that repair and for the relaxed level-$k+2$ repair. The tighter level-k test instead checks all owners. By Thm. 7, a witness at $k+1$ certifies the new optimum by the one-level bound. Otherwise, $k+1$ is excluded, and the cached score or a level-k witness returned by Path-Repair certifies k. If level k is also excluded, the fixed-set update bound ensures that the old answer still attains $k-1$. At $k = 0$, R supplies the nonnegative lower bound. These cases determine the new optimum exactly.

If the level rises, the one-level bound excludes $k+2$, so its repair supplies an upper orientation. If it remains k, the first repair already excludes $k+1$. Transfers in a subsequent level-k repair fill targets only to the tighter budgets, preserving that upper certificate. If the level drops to $k-1$, successful repair at level k gives the required upper orientation. Thus, before answer extraction, x is feasible at level $k'+1$ in every branch.

Complete-root extraction transfers tokens only to owners with spare capacity at boundary $k'-1$. Filling a target to this tighter budget cannot violate its level-$(k'+1)$ upper budget. All intermediate loads are unchanged and root loads decrease. The upper certificate is therefore preserved, including $k' = 0$, when a negative-budget owner is never a spare-capacity target. Each integral transfer strictly decreases total overload. Since a maximum-excess answer has positive excess at the optimum boundary, not all overload can be eliminated. The final complete multi-source closure attains $D_{k'-1}(x)$ and

TABLE I
DATASET STATISTICS AND R-SUBHYPERGRAPH DENSITY COMPARISON

	Dataset statistics				I-ADSH vs. ADSH		
Dataset	$\|V\|$	$\|\mathcal{E}\|$	P	r	Mean gap	Max gap	Matches
Walmart	88,860	65,979	452,440	25	0.125509	0.383333	19/100
Trivago	172,738	220,971	701,952	85	0.032612	0.483333	58/100
Math	176,445	535,323	1,397,229	21	0.016861	0.163636	69/100
Congress	1,718	104,039	2,206,825	400	0.012238	0.139683	74/100
DBLP	1,930,378	2,170,260	7,451,608	280	0.040981	0.263158	56/100
Tags	49,998	5,517,054	21,365,171	5	0.002185	0.095238	96/100
Amazon	2,268,231	4,242,421	72,855,952	9,350	0.030157	0.666667	90/100

hence maximizes excess. Thm. 1 proves integer optimality and containment. Membership and numerator updates restore the remaining state records.

By Lem. 2, the first repair at level $k+1$ starts with at most two overload units after insertion and $|e \setminus R| \le n$ after deletion. If the optimum rises and the algorithm continues at level $k+2$, increasing the budgets cannot create additional overload. Thus, the first two repair calls together perform $O(n)$ augmentations.

For any orientation satisfying $L_x(u) \le B_\gamma(u)$, decreasing the budget parameter from γ to $\gamma - 1$ creates at most one overload unit per ordinary owner and $|A|$ units at the bank. Hence,

$$\sum_{u \in \mathcal{O}} \max\{0, L_x(u) - B_{\gamma-1}(u)\} \le (n - |A|) + |A| = n.$$

The optional level-k repair starts from an orientation feasible at level $k+1$, so it performs at most n augmentations. Likewise, Excess-Closure starts from an orientation feasible at level $k'+1$ and uses boundary budgets $B_{k'-1}$, requiring at most n augmentations. This argument also covers $k' = 0$.

Each augmentation requires one residual search, and each call adds at most one final unsuccessful search. Thus, the complete update performs $O(n)$ searches. Each BFS visits every reached owner and hyperedge at most once and inspects at most H held-edge records and P pins, taking $O(n+P+H)$ time. A path transfer takes $O(n)$ time. Update preparation, root maintenance, answer construction, and numerator recomputation are absorbed by the same total bound, giving $O(n(n + P + H))$ update time. □

V. EXPERIMENTS

A. Experiment settings

Different algorithms. For the static I-ADSH problem, we implement CB-Global (Alg. 1) and CB-Local (Alg. 2). The static baselines are adapted from the global and strongly local flow-based algorithms of Huang et al. [6] using $\epsilon = 1$ and imposing the same mandatory anchor constraint $A \subseteq S$ for each reference set R. We consider both their original real-valued versions, i.e., HGV-Global and HGV-Local, and integer adaptations, denoted by HGV-Global-I and HGV-Local-I. The former yield exact I-ADSH levels through Fact 1; the latter retain the respective flow formulations but terminate upon certifying the optimum integer level, without further optimization of the real-valued density. For dynamic maintenance,

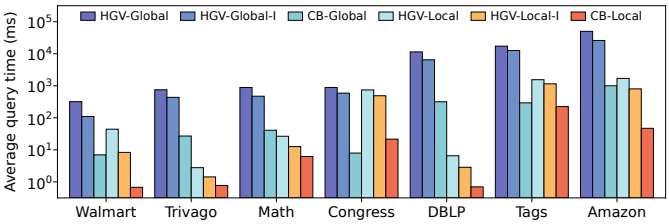


Fig. 5. Running-time comparison of static algorithms.

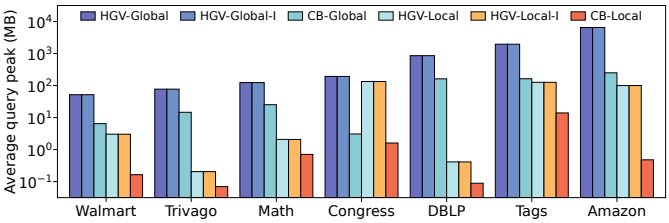


Fig. 6. Peak memory comparison of static algorithms.

we implement Dynamic-Path (Alg. 4). Exact static recomputation with HGV-Local-I and CB-Local after each hyperedge insertion or deletion provides the baselines for evaluating maintenance efficiency. All maximum-flow computations use Dinic's algorithm. All implementations use C++17 and are compiled with GCC using the `-O3` optimization. Experiments run in an Ubuntu 24.04 LTS container on an Intel Xeon Platinum 8490H server, with 64 GiB of memory allocated to the experimental environment.

Datasets. We use seven real-world datasets from Austin R. Benson's hypergraph collection (https://www.cs.cornell.edu/~arb/data/). Their statistics are summarized in Tab. I, where $P = \sum_{e \in \mathcal{E}} |e|$ is the number of vertex–hyperedge incidences and r is the maximum hyperedge size. All hypergraphs are treated as undirected and unweighted, with repeated pins, duplicate hyperedges, and singleton hyperedges removed.

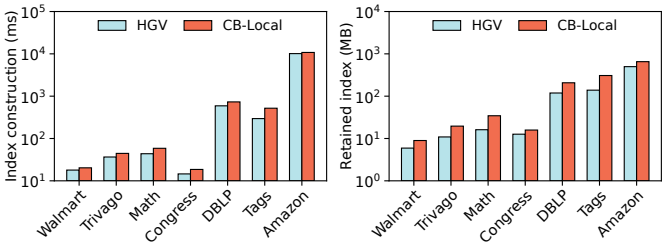
Query generation. For each dataset, we generate 100 queries, adapting the local seed selection and random-walk approach used in anchored-density studies [11], [6]. By default, each query uses four mandatory anchors and a reference set of 64 vertices. We first sample a vertex v uniformly from those incident to a hyperedge of size at most 64, and then sample one such incident hyperedge e_0. The anchor set contains v and is completed by sampling other pins of e_0; vertices within two hypergraph steps of v are used if e_0 has too few pins. We initialize R with $A \cup e_0$, ensuring that R contains an internal hyperedge under all-pins semantics. We then expand R through rounds of three length-two random walks from each anchor. At each step, we sample an incident hyperedge uniformly and then one of its other pins uniformly. An expansion vertex outside $A \cup e_0$ is rejected if its incident-hyperedge count exceeds the smaller of $10 \max_{u \in A} d(u)$ and the 99th percentile of positive incident-hyperedge counts. Candidates that cannot produce exactly 64 reference vertices are resampled.

Remark. In our experiments, we set the maximum number of target owners collected per batched BFS to $4|A|$, clipped to

TABLE II
LOCAL ACCESS FRACTIONS (%)

Dataset	Vertices (%)			Hyperedges (%)			Pins (%)		
	CB	HGV	HGV-I	CB	HGV	HGV-I	CB	HGV	HGV-I
Walmart	**1.454**	9.642	9.624	2.517	2.522	**2.510**	**0.981**	4.039	4.031
Trivago	**0.166**	0.280	0.279	0.162	**0.157**	**0.157**	**0.140**	0.201	0.200
Math	**2.111**	2.804	2.795	0.897	0.900	**0.889**	**0.840**	1.044	1.035
Congress	**67.106**	93.066	93.066	38.377	**38.376**	**38.376**	**8.641**	68.854	68.854
DBLP	**0.030**	0.056	0.056	0.028	0.028	**0.027**	**0.022**	0.034	0.034
Tags	**25.656**	38.679	38.679	5.501	**5.500**	**5.500**	**3.871**	5.833	5.833
Amazon	**0.269**	13.606	13.606	**0.232**	**0.232**	**0.232**	**0.033**	0.867	0.867

Note: CB = CB-Local, HGV = HGV-Local, and HGV-I = HGV-Local-I.



(a) Construction time (b) Retained index

Fig. 7. Index construction time and retained memory.

$[8, 64]$, in both CB-Global and CB-Local. Each target owner is the endpoint of a candidate token-transfer path.

B. Performance studies

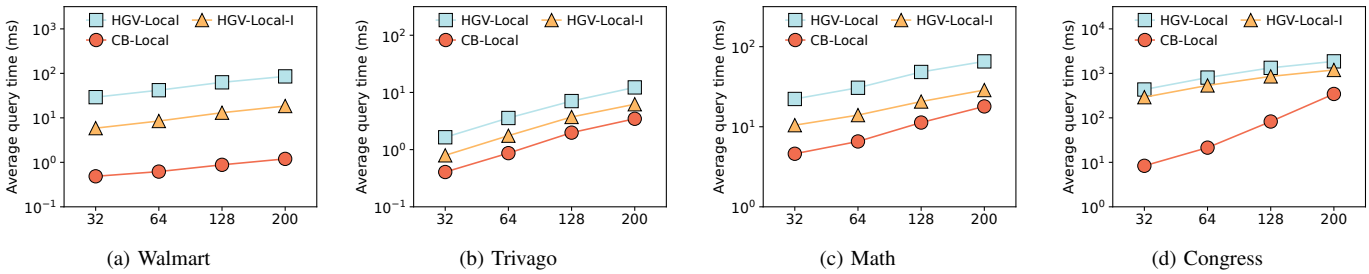
Exp-1: Running-time comparison of static algorithms. We compare CB-Global, CB-Local, HGV-Global, HGV-Local, HGV-Global-I, and HGV-Local-I using the same 100 queries per dataset. Each query is executed three times, and its running time is the median of the three repetitions. Fig. 5 shows the mean query time over the 100 queries for each algorithm on each dataset. Across these datasets, CB-Global achieves speedups of 21.61–111.46× over HGV-Global and 11.57–74.30× over HGV-Global-I. Likewise, CB-Local is 3.59–65.25× faster than HGV-Local and 1.85–22.63× faster than HGV-Local-I. The global-flow baselines are slower than their local counterparts on every dataset: HGV-Global is 1.19–1,743.92× slower than HGV-Local, while HGV-Global-I is 1.21–2,269.24× slower than HGV-Local-I. The same trend holds for the proposed methods on all datasets except Congress: CB-Global is 1.31–454.03× slower than CB-Local, whereas on Congress, CB-Global is 2.73× faster. This exception arises because Congress has a small vertex set and a high incidence volume, causing the preparation overhead of CB-Local to outweigh the benefits of local exploration. Overall, these results demonstrate the efficiency of the proposed static algorithms.

Exp-2: Peak memory comparison of static algorithms. Using the same 100 queries as in Exp-1, we evaluate the six static algorithms by measuring, for each query, the maximum additional memory allocated during execution, excluding shared graph and index memory, and report the mean per-query peak in Fig. 6. Across all seven datasets, the Local variants use less peak memory than their Global counterparts: CB-Local is 1.92–1,821.58× smaller than CB-Global, HGV-Local is 1.44–2,071.61× smaller than HGV-Global, and HGV-Local-I is 1.44–2,080.03× smaller than HGV-Global-I. CB-Local is 1.85–222.01× smaller than HGV-Local and 1.84–222.01× smaller than HGV-Local-I. Similarly, CB-Global is 3.35–62.62× smaller than HGV-Global and 3.35–62.62× smaller than HGV-Global-I. Overall, these results confirm that the local variants consistently require less peak memory than their corresponding global methods.

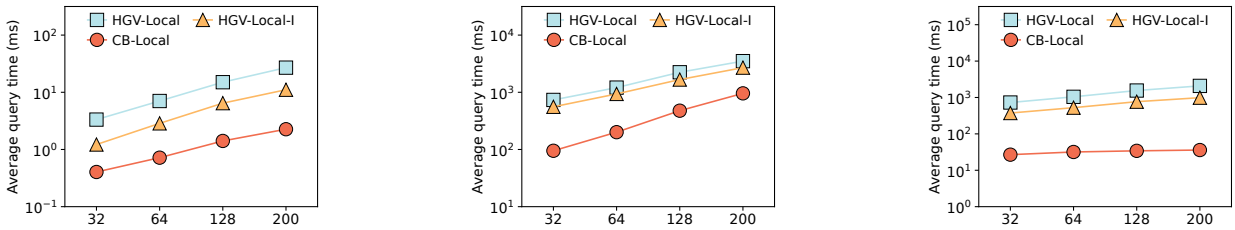
Exp-3: Local access. We compare HGV-Local, HGV-Local-I, and CB-Local on the same 100 queries as Exp-1. For each complete query, we count the distinct original vertices, hyperedges, and pins accessed during preparation, search/flow, token transfer, and answer extraction. We normalize these counts by $|V|$, $|\mathcal{E}|$, and P and average them over the 100 queries. Tab. II reports these complete-query fractions. CB-Local has smaller mean vertex and pin access fractions on all seven datasets, while hyperedge access fractions are often similar. For example, on Congress, it accesses 67.106% of vertices and 8.641% of pins, compared with 93.066% and 68.854% for both HGV-Local and HGV-Local-I; on Amazon, the corresponding fractions are 0.269% and 0.033% for CB-Local, versus 13.606% and 0.867% for the two HGV variants. These results confirm that CB-Local substantially reduces vertex and pin accesses while keeping hyperedge access comparable.

Exp-4: CB-Local index evaluation. We compare CB-Local, HGV-Local, and HGV-Local-I in this experiment. HGV-Local and HGV-Local-I share the same base preprocessing and retain adjacency, degree, and fractional-degree data. CB-Local retains these base data plus token assignments, loads, and held-edge records. Because the two HGV variants share the same retained structure, one HGV bar represents both. Fig. 7 compares the CB retained structure with this common HGV base; the difference between the bars gives the additional CB index size. Complete CB preparation takes 18.583–10,792.738 ms, and its additional index ranges from 3.031 to 242.188 MB. For example, on Amazon, CB-Local takes 10,792.738 ms and retains 653.907 MB, compared with 10,133.182 ms and 498.480 MB for HGV; thus, CB-Local requires 1.07× the construction time and 1.31× the retained memory. These results show that CB-Local incurs modest construction and memory costs for its index while delivering the query-time gains reported in Exp. 1.

Exp-5: Sensitivity to reference-set size. Following the reference-set sensitivity studies of ADS and AADS [11], [13], we vary $|R| \in \{32, 64, 128, 200\}$ while fixing $|A| = 4$. For each dataset, we generate 100 queries with $|A \cup e_0| \leq 32$ and construct nested reference sets $R_{32} \subseteq R_{64} \subseteq R_{128} \subseteq R_{200}$ by taking prefixes of a 200-vertex sequence. Fig. 8 shows mean query time for all seven datasets. In all seven datasets, the query times of CB-Local, HGV-Local, and HGV-Local-I increase as $|R|$ grows from 32 to 200. Across all reference-set sizes, CB-Local is significantly faster than both HGV-Local and HGV-Local-I, consistent with the results in Exp. 1. These results show that CB-Local remains efficient as $|R|$ grows.

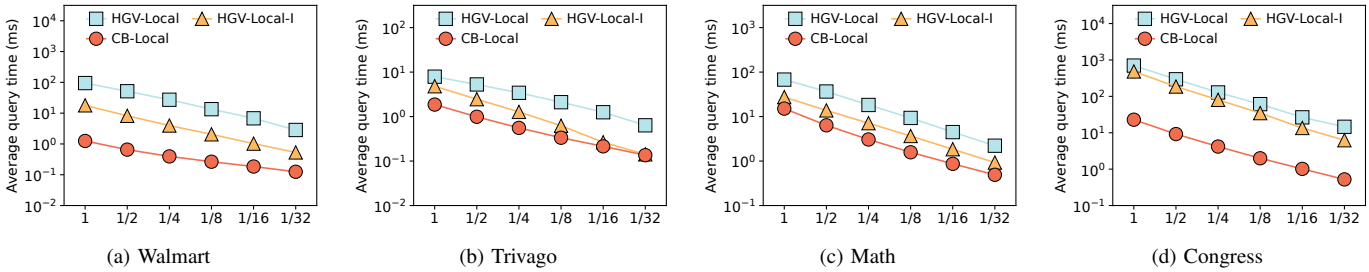


(a) Walmart (b) Trivago (c) Math (d) Congress

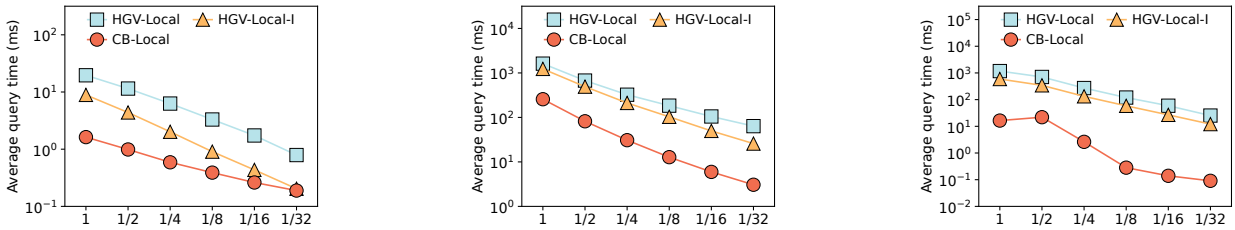


(e) DBLP (f) Tags (g) Amazon

Fig. 8. Sensitivity test of reference-set size for the Local methods.



(a) Walmart (b) Trivago (c) Math (d) Congress



(e) DBLP (f) Tags (g) Amazon

Fig. 9. Sensitivity test of hyperedge retention for the Local methods.

Exp-6: Sensitivity to hyperedge density. We adopt the edge-sampling procedure used in ADS and AADS [11], [13] to hypergraphs to study sensitivity to hyperedge density. Each hyperedge is independently retained with probability $p \in \{1, 1/2, 1/4, 1/8, 1/16, 1/32\}$. Reusing the same random samples across different values of p produces nested snapshots, with smaller p yielding sparser snapshots. We fix $|A| = 4$ and $|R| = 64$, generate 100 queries on the sparsest snapshot $\mathcal{H}_{1/32}$ using the default procedure, and reuse the same queries on every denser snapshot and with all three Local methods. Fig. 9 shows mean query time for all seven datasets. Overall, reducing p lowers the query times because the sampled hypergraphs contain fewer incidences, although the per-dataset trend need not be strictly monotone. For Amazon, CB-Local takes 16.41 ms at $p = 1$ and 21.97 ms at $p = 1/2$; this local increase is due to query-specific preparation overhead, which dominates the solver time, and the query time decreases at lower retention probabilities. In all seven datasets, CB-Local is significantly faster than HGV-Local and HGV-Local-I at every retention probability, consistent with the results in Exp. 1. These results show that CB-Local remains efficient across hypergraph densities.

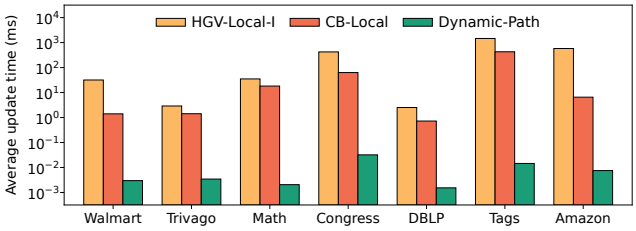


Fig. 10. Running-time comparison under random hyperedge insertion.

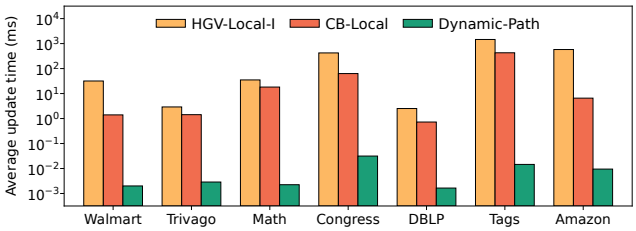


Fig. 11. Running-time comparison under random hyperedge deletion.

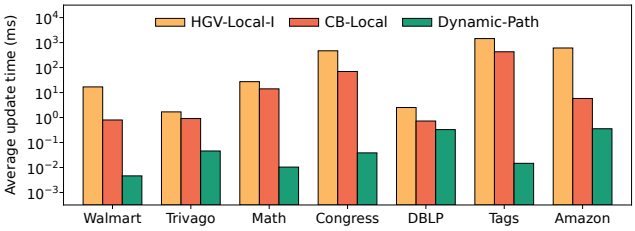


Fig. 12. Running-time comparison under query-local hyperedge insertion.

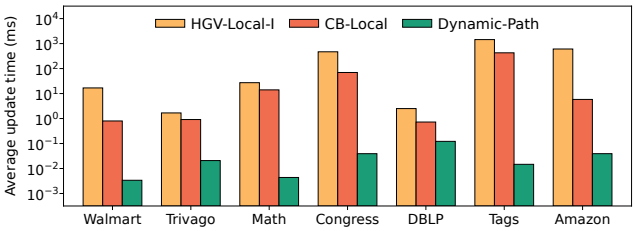


Fig. 13. Running-time comparison under query-local hyperedge deletion.

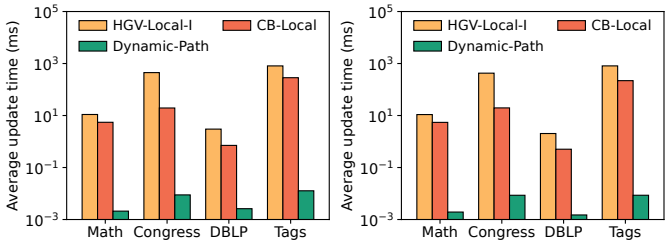
Exp-7: Running-time comparison of dynamic algorithms. We compare running times of Dynamic-Path, CB-Local, and HGV-Local-I under the two workloads. For the random-global workload, we select the ten verified Exp-1 queries with the highest initial integer levels and randomly sample 1,000 hyperedges for each query. For the query-local workload, we use the same queries and select 1,000 hyperedges from the incidence neighborhood of each query's reference set R. Each selected hyperedge is deleted and then immediately reinserted, yielding 2,000 update events per query. We report the mean time per update over the ten queries, with deletions and insertions measured separately. Fig. 10 and Fig. 11 show the random-global results, while Fig. 12 and Fig. 13 report the query-local results. Dynamic-Path is the fastest method on every dataset under both workloads and for both update directions; CB-Local is consistently second, followed by HGV-Local-I. Across the 28 dataset–workload–operation combinations, Dynamic-Path is faster than CB-Local by roughly three orders of magnitude and faster than HGV-Local-I by roughly four orders of magnitude in geometric mean across all datasets. Consistent with Exp. 1, CB-Local is $6.73\times$ faster than HGV-Local-I. Query-local updates can increase the repair cost of Dynamic-Path on some datasets, most visibly DBLP and Amazon, but do not change this ranking.

In addition, we evaluate real temporal insertions and deletions on datasets with timestamps. We select the 1,000 most recent hyperedges by timestamp on the four timestamped datasets: Math, Congress, DBLP, and Tags. For insertion, we start from the snapshot before these hyperedges and add them in chronological order; for deletion, we start from the complete hypergraph and remove the same hyperedges in reverse chronological order. Fig. 14 shows the temporal update times. Dynamic-Path again has the lowest mean update time on all four datasets for both directions. Across the dataset–direction pairs, it is roughly two to four orders of magnitude faster than CB-Local and roughly three to five orders of magnitude faster than HGV-Local-I. Taken together, the results across all three update settings demonstrate that Dynamic-Path provides consistently efficient and robust dynamic maintenance across diverse update regimes, while the flow-based baselines incur substantially higher update costs.

C. Answer quality and case studies

Exp-8: R-subhypergraph density comparison between I-ADSH and ADSH. We compare the complete answers produced by I-ADSH and ADSH for all 700 Exp-1 queries, focusing on their R-subhypergraph densities and returned vertex sets. For each query q, let $\widehat{S}_{1,q}$ and $S^*_{I,q}$ denote the ADSH answer at $\epsilon = 1$ and the corresponding I-ADSH answer, respectively. Using the unrounded two-unit density ρ_R introduced in Def. 3, define $\Delta_q = \rho_R(\widehat{S}_{1,q}) - \rho_R(S^*_{I,q})$. We compute each rational gap before averaging, and separately count exact equality of densities and full vertex sets. For every query, $S^*_{I,q}$ contains the vertex set of the real-valued ADSH solution $\widehat{S}_{1,q}$. Exactly 462 queries (66.00%) match that ADSH solution in both density and vertex set. The other 238 queries have strictly lower unrounded density for $S^*_{I,q}$. Tab. I reports per-dataset density gaps and equality counts. The largest observed gap is 0.667, below the theoretical bound of one. These results confirm that I-ADSH provides an approximate solution to ADSH with a closely matching R-subhypergraph density.

Exp-9: Case study on DBLP-DB. We construct DBLP-DB, a database-research collaboration hypergraph, from the official DBLP snapshot dated October 3, 2026 [14]. We include publications from 2015–2025 in SIGMOD, VLDB, ICDE, PODS, EDBT, ICDT, TODS, PVLDB, the VLDB Journal, and PACMMOD, excluding workshops and companion venues. Each distinct complete author set contributes one unit-weight hyperedge; singleton publications retain their vertices but do not contribute an optimization hyperedge. We obtain 19,451 authors, 9,993 hyperedges, and 49,450 incidences, with a



(a) Insertion (b) Deletion

Fig. 14. Running-time comparison on temporal hypergraphs.

maximum hyperedge size of 44. We compare HGV-Local and CB-Local on ten queries centered at Xuemin Lin, generated via random-walks in Sec. V-A with $|A| = 4$ and $|R| = 64$. The instance in Fig. 15 uses Ying Zhang, Xiang Wang, Wenjie Zhang, and Xuemin Lin as anchors.

As seen, HGV-Local returns 14 authors and 13 hyperedges, whereas CB-Local obtains 17 authors and 15 hyperedges. Both outputs are connected, lie within R, and attain the optimal integer level 2. CB-Local contains the real-valued-optimal set returned by HGV-Local and adds Boge Liu, Fan Zhang, and Qingyuan Linghu. The added hyperedges, E07 and E08, correspond to *Efficient Community Search with Size Constraint* (ICDE 2021) [15] and *Global Reinforcement of Social Networks: The Anchored Coreness Problem* (SIGMOD 2020) [16], respectively. With zero non-reference penalty, these two added hyperedges increase N_R by four while adding three authors, raising the total excess at boundary 1 from $26 - 14 = 12$ to $30 - 17 = 13$. The resulting unrounded densities are $13/7$ and $30/17$, respectively, yielding a relative gap of $11/221 \approx 4.98\%$. This difference reflects the objectives: CB-Local selects the larger team with greater boundary excess, whereas HGV-Local selects the team with maximum unrounded density.

VI. Related Work

Graph mining and community discovery. Community discovery studies query-centered cohesion and seeded partitioning [17], [18], [19], [20], [21], [22], [23]. Densest-subgraph algorithms use flow, peeling, core pruning, and scalable processing [10], [8], [24], [9], [25], [26], [27], [28]. Extensions address size constraints, directions, multiple layers, and robustness [29], [30], [31], [32], [33], or use quasi-clique and clique-based density [34], [35], [36]. Density-friendly decompositions and locally densest models expose multiple dense regions [37], [38], [39], [40]. Orientations and path reversals support load balancing [41], [42], [43], [44], while dynamic algorithms maintain approximate density [45], [46], [47]. For query-aware density, ADS penalizes non-reference vertices by degree [11], whereas NR-density uses within-result neighborhood penalties [12]. Integer AADS uses orientations for static optimization and dynamic maintenance [13]. Our objective reduces to integer graph AADS at rank two; an anchor-bank orientation handles complete-hyperedge rewards and incidence penalties for exact search and update repair.

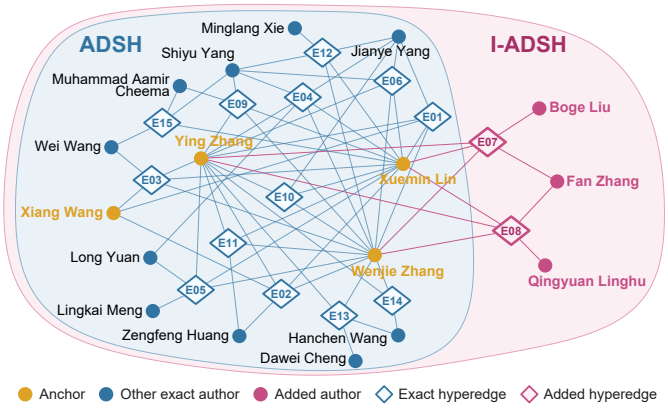


Fig. 15. An example of anchored collaboration teams on DBLP-DB.

Higher-order network analysis and hypergraph mining. Higher-order analysis studies motif organization and simplicial closure [3], [2], [48], spectral learning and graph representations [4], [5], [49], [50], and local clustering and random walks [51], [52], [53]. Hypergraph communities are also modeled through categorical labels, modularity, and cores [54], [55], [56], [57], [58], [59]. Cut models range from flow reductions to submodular splitting functions [60], [61], [62], [63], [64], with localized ratio-cut, PageRank, and diffusion methods supporting seeded clustering [65], [66], [67], [68]. Bera et al. maintain approximate global hypergraph density [7]. Closest to our setting, Huang et al. introduce ADSH with tunable locality and global/local flow algorithms [6]. Their local method expands the explored region as needed and can solve several flow networks within one query. We optimize its integer counterpart at $\epsilon = 1$ with mandatory anchors; our orientation certificate enables direct balancing, reusable-index local search, and exact hyperedge-update repair.

VII. Conclusion

In this paper, we formulate exact integer anchored densest-subhypergraph search with an output criterion that maximizes total excess at the optimal integer boundary. We establish an anchor-bank orientation framework that characterizes the optimum integer level through threshold and min–max views, and develop CB-Global and CB-Local to compute it; CB-Local reuses a query-independent orientation index across queries. For dynamic maintenance, we prove that a single hyperedge update changes the optimum level by at most one and exploit this property in Dynamic-Path to restore the exact level and maximum-excess solution without rerunning a static solver. Experiments on real hypergraphs demonstrate the practical effectiveness and efficiency of our static and dynamic methods.

References

[1] M. E. J. Newman, *Networks*, 2nd ed. Oxford University Press, 2018.
[2] A. R. Benson, D. F. Gleich, and J. Leskovec, "Higher-order organization of complex networks," *Science*, vol. 353, no. 6295, pp. 163–166, 2016.
[3] F. Battiston, G. Cencetti, I. Iacopini, V. Latora, M. Lucas, A. Patania, J.-G. Young, and G. Petri, "Networks beyond pairwise interactions: Structure and dynamics," *Phys. Rep.*, vol. 874, pp. 1–92, 2020.
[4] A. Bretto, *Hypergraph Theory: An Introduction*. Springer, 2013.

[5] D. Zhou, J. Huang, and B. Schölkopf, "Learning with hypergraphs: Clustering, classification, and embedding," in *NeurIPS*, 2006, pp. 1601–1608.

[6] Y. Huang, D. F. Gleich, and N. Veldt, "Densest subhypergraph: Negative supermodular functions and strongly localized methods," in *WWW*, 2024, pp. 881–892.

[7] S. K. Bera, S. Bhattacharya, J. Choudhari, and P. Ghosh, "A new dynamic algorithm for densest subhypergraphs," in *WWW*, 2022, pp. 1093–1103.

[8] A. V. Goldberg, "Finding a maximum density subgraph," University of California, Berkeley, Tech. Rep. UCB/CSD-84-171, 1984.

[9] M. Charikar, "Greedy approximation algorithms for finding dense components in a graph," in *APPROX*, 2000, pp. 84–95.

[10] T. Lanciano, A. Miyauchi, A. Fazzone, and F. Bonchi, "A survey on the densest subgraph problem and its variants," *ACM Comput. Surv.*, vol. 56, no. 8, 2024.

[11] Y. Dai, M. Qiao, and L. Chang, "Anchored densest subgraph," in *SIGMOD*, 2022, pp. 1200–1213.

[12] X. Ye, R.-H. Li, L. Liang, Z. Liu, L. Lin, and G. Wang, "Efficient and effective anchored densest subgraph search: A convex-programming based approach," in *KDD*, 2024, pp. 3907–3918.

[13] Q. Zhang, Y. Zhang, R.-H. Li, and G. Wang, "Approximate anchored densest subgraph search on large static and dynamic graphs," *Proc. VLDB Endow.*, vol. 18, no. 3, pp. 623–636, 2024.

[14] dblp Team, "dblp computer science bibliography – monthly snapshot XML release of october 2026," Schloss Dagstuhl–Leibniz Center for Informatics, 2026.

[15] B. Liu, F. Zhang, W. Zhang, X. Lin, and Y. Zhang, "Efficient community search with size constraint," in *ICDE*, 2021, pp. 97–108.

[16] Q. Linghu, F. Zhang, X. Lin, W. Zhang, and Y. Zhang, "Global reinforcement of social networks: The anchored coreness problem," in *SIGMOD*, 2020, pp. 2211–2226.

[17] Y. Fang, X. Huang, L. Qin, Y. Zhang, W. Zhang, R. Cheng, and X. Lin, "A survey of community search over big graphs," *VLDB J.*, vol. 29, no. 1, pp. 353–392, 2020.

[18] M. Sozio and A. Gionis, "The community-search problem and how to plan a successful cocktail party," in *KDD*, 2010, pp. 939–948.

[19] W. Cui, Y. Xiao, H. Wang, and W. Wang, "Local search of communities in large graphs," in *SIGMOD*, 2014, pp. 991–1002.

[20] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, "Querying k-truss community in large and dynamic graphs," in *SIGMOD*, 2014, pp. 1311–1322.

[21] R. Andersen, F. Chung, and K. Lang, "Local graph partitioning using PageRank vectors," in *FOCS*, 2006, pp. 475–486.

[22] L. Orecchia and Z. A. Zhu, "Flow-based algorithms for local graph clustering," in *SODA*, 2014, pp. 1267–1286.

[23] N. Veldt, C. Klymko, and D. F. Gleich, "Flow-based local graph clustering with better seed set inclusion," in *SDM*, 2019, pp. 378–386.

[24] G. Gallo, M. D. Grigoriadis, and R. E. Tarjan, "A fast parametric maximum flow algorithm and applications," *SIAM J. Comput.*, vol. 18, no. 1, pp. 30–55, 1989.

[25] C. Chekuri, K. Quanrud, and M. R. Torres, "Densest subgraph: Supermodularity, iterative peeling, and flow," in *SODA*, 2022, pp. 1531–1555.

[26] Y. Fang, K. Yu, R. Cheng, L. V. S. Lakshmanan, and X. Lin, "Efficient algorithms for densest subgraph discovery," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1719–1732, 2019.

[27] B. Bahmani, R. Kumar, and S. Vassilvitskii, "Densest subgraph in streaming and MapReduce," *Proc. VLDB Endow.*, vol. 5, no. 5, pp. 454–465, 2012.

[28] L. Chang and M. Qiao, "Deconstruct densest subgraphs," in *WWW*, 2020, pp. 2747–2753.

[29] R. Andersen and K. Chellapilla, "Finding dense subgraphs with size bounds," in *WAW*, 2009, pp. 25–37.

[30] S. Khuller and B. Saha, "On finding dense subgraphs," in *ICALP*, 2009, pp. 597–608.

[31] C. Ma, Y. Fang, R. Cheng, L. V. S. Lakshmanan, W. Zhang, and X. Lin, "Efficient algorithms for densest subgraph discovery on large directed graphs," in *SIGMOD*, 2020, pp. 1051–1066.

[32] E. Galimberti, F. Bonchi, and F. Gullo, "Core decomposition and densest subgraph in multilayer networks," in *CIKM*, 2017, pp. 1807–1816.

[33] A. Miyauchi and A. Takeda, "Robust densest subgraph discovery," in *ICDM*, 2018, pp. 1188–1193.

[34] C. Tsourakakis, F. Bonchi, A. Gionis, F. Gullo, and M. Tsiarli, "Denser than the densest subgraph: Extracting optimal quasi-cliques with quality guarantees," in *KDD*, 2013, pp. 104–112.

[35] C. Tsourakakis, "The K-clique densest subgraph problem," in *WWW*, 2015, pp. 1122–1132.

[36] B. Sun, M. Danisch, T.-H. H. Chan, and M. Sozio, "KClist++: A simple algorithm for finding k-clique densest subgraphs in large graphs," *Proc. VLDB Endow.*, vol. 13, no. 10, pp. 1628–1640, 2020.

[37] N. Tatti, "Density-friendly graph decomposition," *ACM Trans. Knowl. Discov. Data*, vol. 13, no. 5, pp. 1–29, 2019.

[38] M. Danisch, T.-H. H. Chan, and M. Sozio, "Large scale density-friendly graph decomposition via convex programming," in *WWW*, 2017, pp. 233–242.

[39] L. Qin, R.-H. Li, L. Chang, and C. Zhang, "Locally densest subgraph discovery," in *KDD*, 2015, pp. 965–974.

[40] T. B. Trung, L. Chang, N. T. Long, K. Yao, and H. T. Thanh Binh, "Verification-free approaches to efficient locally densest subgraph discovery," in *ICDE*, 2023, pp. 1–13.

[41] Y. Asahiro, E. Miyano, H. Ono, and K. Zenmyo, "Graph orientation algorithms to minimize the maximum outdegree," *Int. J. Found. Comput. Sci.*, vol. 18, no. 2, pp. 197–215, 2007.

[42] Ł. Kowalik, "Approximation scheme for lowest outdegree orientation and graph density measures," in *ISAAC*, 2006, pp. 557–566.

[43] M. Blumenstock, "Fast algorithms for pseudoarboricity," in *ALENEX*, 2016, pp. 113–126.

[44] G. Borradaile, J. Iglesias, T. Migler, A. Ochoa, G. Wilfong, and L. Zhang, "Egalitarian graph orientations," *J. Graph Algorithms Appl.*, vol. 21, no. 4, pp. 687–708, 2017.

[45] A. Epasto, S. Lattanzi, and M. Sozio, "Efficient densest subgraph computation in evolving graphs," in *WWW*, 2015, pp. 300–310.

[46] S. Bhattacharya, M. Henzinger, D. Nanongkai, and C. Tsourakakis, "Space- and time-efficient algorithm for maintaining dense subgraphs on one-pass dynamic streams," in *STOC*, 2015, pp. 173–182.

[47] S. Sawlani and J. Wang, "Near-optimal fully dynamic densest subgraph," in *STOC*, 2020, pp. 181–193.

[48] A. R. Benson, R. Abebe, M. T. Schaub, A. Jadbabaie, and J. Kleinberg, "Simplicial closure and higher-order link prediction," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 48, pp. E11 221–E11 230, 2018.

[49] S. Agarwal, K. Branson, and S. Belongie, "Higher order learning with graphs," in *ICML*, 2006, pp. 17–24.

[50] A. R. Benson, D. F. Gleich, and J. Leskovec, "Tensor spectral clustering for partitioning higher-order network structures," in *SDM*, 2015, pp. 118–126.

[51] H. Yin, A. R. Benson, J. Leskovec, and D. F. Gleich, "Local higher-order graph clustering," in *KDD*, 2017, pp. 555–564.

[52] U. Chitra and B. Raphael, "Random walks on hypergraphs with edge-dependent vertex weights," in *ICML*, 2019, pp. 1172–1181.

[53] T. Carletti, F. Battiston, G. Cencetti, and D. Fanelli, "Random walks on hypergraphs," *Phys. Rev. E*, vol. 101, no. 2, p. 022308, 2020.

[54] I. Amburg, N. Veldt, and A. R. Benson, "Clustering in graphs and hypergraphs with categorical edge labels," in *WWW*, 2020, pp. 706–717.

[55] B. Kamiński, V. Poulin, P. Prałat, P. Szufel, and F. Théberge, "Clustering via hypergraph modularity," *PLoS One*, vol. 14, no. 11, p. e0224307, 2019.

[56] P. S. Chodrow, N. Veldt, and A. R. Benson, "Generative hypergraph clustering: From blockmodels to modularity," *Sci. Adv.*, vol. 7, no. 28, p. eabh1303, 2021.

[57] N. A. Arafat, A. Khan, A. K. Rai, and B. Ghosh, "Neighborhood-based hypergraph core decomposition," *Proc. VLDB Endow.*, vol. 16, no. 9, pp. 2061–2074, 2023.

[58] J. Lee, K.-I. Goh, D.-S. Lee, and B. Kahng, "(k, q)-core decomposition of hypergraphs," *Chaos Solitons Fractals*, vol. 173, p. 113645, 2023.

[59] F. Bu, G. Lee, and K. Shin, "Hypercore decomposition for non-fragile hyperedges: Concepts, algorithms, observations, and applications," *Data Min. Knowl. Discov.*, vol. 37, no. 6, pp. 2389–2437, 2023.

[60] E. L. Lawler, "Cutsets and partitions of hypergraphs," *Networks*, vol. 3, no. 3, pp. 275–285, 1973.

[61] P. Li and O. Milenkovic, "Inhomogeneous hypergraph clustering with applications," in *NeurIPS*, 2017, pp. 2308–2318.

[62] ——, "Submodular hypergraphs: p-laplacians, cheeger inequalities and spectral clustering," in *ICML*, 2018, pp. 3014–3023.

[63] M. Hein, S. Setzer, L. Jost, and S. S. Rangapuram, "The total variation on hypergraphs - learning on hypergraphs revisited," in *NeurIPS*, 2013, pp. 2427–2435.

[64] N. Veldt, A. R. Benson, and J. Kleinberg, "Hypergraph cuts with general splitting functions," *SIAM Rev.*, vol. 64, no. 3, pp. 650–685, 2022.
[65] ——, "Minimizing localized ratio cut objectives in hypergraphs," in *KDD*, 2020, pp. 1708–1718.
[66] Y. Takai, A. Miyauchi, M. Ikeda, and Y. Yoshida, "Hypergraph clustering based on PageRank," in *KDD*, 2020, pp. 1970–1978.
[67] M. Liu, N. Veldt, H. Song, P. Li, and D. F. Gleich, "Strongly local hypergraph diffusions for clustering and semi-supervised learning," in *WWW*, 2021, pp. 2092–2103.
[68] K. Fountoulakis, P. Li, and S. Yang, "Local hyper-flow diffusion," in *NeurIPS*, 2021, pp. 27 683–27 694.